

Sviluppo di un Simulatore Predittivo per la FIFA World Cup 2026

Studente: Angelo Lapacciana

Corso di Metodi Matematici per l'Intelligenza Artificiale

Università degli Studi di Bari

Luglio 2026

Sommario

Il seguente elaborato descrive lo sviluppo di un sistema software per prevedere e simulare i risultati della **FIFA World Cup 2026**. Data la *stocasticità* di uno sport di squadra come il calcio, l'obiettivo principale è stato creare un modello capace di gestire questa incertezza, fornendo probabilità realistiche anziché previsioni "secche" su chi vincerà una partita.

Per farlo, sono stati combinati i dati storici delle partite internazionali con la metrica di **ranking Elo** delle nazionali. Un passaggio fondamentale è stato l'**allineamento temporale** dei dati: per evitare che il modello "barasse" guardando informazioni future (il cosiddetto *data leakage*), i punteggi Elo e lo *stato di forma* sono stati calcolati prima del fischio d'inizio di ogni match analizzato.

Il cuore predittivo del sistema è un **Soft Ensemble** di tre algoritmi di *Machine Learning* (**Random Forest**, **XGBoost** e **LightGBM**). Il modello è stato addestrato rispettando l'ordine cronologico degli eventi (tramite `TimeSeriesSplit`) e ottimizzato usando la funzione di costo **Log Loss**. Inoltre, si è prestata particolare attenzione al **bilanciamento dei dati** per permettere al modello di riconoscere i pareggi, storicamente difficili da prevedere.

Le probabilità generate da questo modello alimentano poi un **simulatore Monte Carlo**. Questo motore replica il nuovo **formato a 48 squadre** della FIFA e simula migliaia di interi mondiali, aggiornando l'Elo delle squadre partita dopo partita e inserendo variazioni casuali per simulare i cali o i picchi di forma.

Tutto il sistema è infine accessibile tramite un'applicazione web interattiva creata con **Streamlit**. L'interfaccia permette sia di calcolare le quote per una singola partita, sia di avviare l'**"Oracolo"** per simulare l'intero torneo in *tempo reale*, restituendo una classifica aggiornata delle reali favorite alla vittoria della Coppa del Mondo.

Indice

1	Introduzione	4
1.1	Contesto e Motivazioni	4
1.2	Obiettivi del Progetto	4
1.3	Struttura dell'Elaborato	5
2	Analisi dei Dati e Feature Engineering	5
2.1	I Dataset di Partenza	5
2.2	Il Problema del Data Leakage e l'Allineamento Temporale	6
2.3	Preprocessing e Costruzione delle Feature Predittive	7
2.4	Analisi Esplorativa dei Dati (EDA)	8
3	Architettura del Modello Predittivo	11
3.1	Definizione del Target e Bilanciamento delle Classi	11
3.2	Gli Algoritmi di Machine Learning	13
3.3	Strategia di Addestramento	15
3.4	Il Soft Ensemble	16
3.5	Valutazione delle Performance	17
4	Il Motore di Simulazione Monte Carlo	21
4.1	Caricamento delle Risorse e Struttura Algoritmica	21
4.2	Simulazione del Singolo Match e Varianza Stocastica	21
4.3	Aggiornamento Dinamico dell'Elo	23
4.4	Logica del Tabellone a Eliminazione Diretta	24
5	Implementazione Software e Deploy	25
5.1	Gestione della Configurazione e dei Dati Live	25
5.2	Sviluppo dell'Interfaccia Streamlit	26
5.3	Modulo 1: Previsore Singolo Match	27
5.4	Modulo 2: L'“Oracolo” (Simulazione Globale)	28
6	Risultati e Discussione	28
6.1	Analisi della Simulazione Globale e Convergenza	28
6.2	Limiti del Modello	30
7	Conclusioni	31
7.1	Sintesi del Lavoro Svolto	31
7.2	Sviluppi Futuri e Ottimizzazioni	32
A	Il Sistema di Rating Elo nel Calcio	33
B	Automazione dell'Aggiornamento Dati tramite API	34
C	Guida all'utilizzo dell'applicazione web	35

1 Introduzione

1.1 Contesto e Motivazioni

Il calcio rappresenta una delle sfide più ardue nel campo della modellazione predittiva. Dal punto di vista statistico, è classificato come uno sport a “basso punteggio” (*low-scoring*). In questa categoria di eventi, la **legge dei grandi numeri** non riesce a esercitare il suo effetto compensativo, come avviene invece negli sport ad alto punteggio (ad esempio, il basket), dove l’elevato volume di azioni diluisce il peso statistico del singolo episodio.

Di conseguenza, il calcio è caratterizzato da un’incertezza intrinseca e da un’alta **varianza**: un singolo evento fortuito (quale un palo, un rimbalzo imprevedibile, un errore arbitrale o un infortunio improvviso) possiede un peso sproporzionato e può alterare radicalmente l’esito finale di un incontro. Questa natura intrinsecamente “rumorosa” rende i modelli deterministici (che tentano di prevedere un unico esito certo) inaffidabili, giustificando la necessità di un approccio **probabilistico** e stocastico come quello proposto in questo elaborato.

Il contesto in cui questo progetto si inserisce è quello della **FIFA World Cup 2026**, che è in corso in Stati Uniti, Messico e Canada. Per la prima volta, il torneo passerà dal formato a 32 squadre a quello a **48 squadre** con conseguente introduzione di 12 gironi da 4 squadre. L’aumento del numero di partecipanti e la modifica della struttura del tabellone aumentano il numero di partite giocate e, di conseguenza, la complessità nel calcolare le reali probabilità di vittoria finale di una nazionale.

1.2 Obiettivi del Progetto

Proprio per questa complessità, l’obiettivo del progetto non è creare una “sfera di cristallo” capace di prevedere il futuro, ma sviluppare un programma basato sulle probabilità. L’idea di fondo è gestire l’incertezza tipica del calcio, trasformando l’imprevedibilità del campo in percentuali matematiche e scenari realistici.

Per raggiungere questo scopo, lo sviluppo del sistema si è basato sulle seguenti scelte principali:

1. **Approccio probabilistico:** Abbandonare l’illusione dei modelli deterministici (“Chi vincerà?”) in favore di una stima distributiva (“Con quale probabilità ogni esito può verificarsi?”), fornendo un output utile sia per l’analisi sportiva che per il calcolo delle quote.
2. **Rigore cronologico (No Data Leakage):** Garantire un addestramento corretto, assicurandosi che il modello utilizzi per le sue stime solo i dati che erano realmente disponibili prima del fischio d’inizio della partita.
3. **Replica dell’ecosistema FIFA:** Integrare un motore di simulazione che replichi fedelmente le regole del Mondiale 2026, tenendo conto dell’evoluzione della forma delle squadre e dell’aggiornamento dinamico della loro forza relativa.
4. **Sviluppo di un tool interattivo:** Sviluppare un’interfaccia grafica intuitiva che permetta agli utenti di interagire con il modello, sia per analizzare il singolo match che per osservare le macro-tendenze dell’intero torneo.

1.3 Struttura dell'Elaborato

Il presente elaborato è strutturato per guidare il lettore attraverso l'intero ciclo di vita del progetto, dall'acquisizione dei dati fino al deploy dell'applicazione. Nello specifico, il lavoro si articola come segue:

- Il **Capitolo 2** introdurrà l'analisi dei dati e la fase di *Feature Engineering*, con un focus cruciale sulla gestione temporale dei dati, evidenziando le criticità legate al data leakage e le soluzioni adottate per mitigarlo.
- Il **Capitolo 3** dettaglierà l'architettura del modello predittivo, spiegando l'evoluzione degli algoritmi scelti, i compromessi affrontati durante l'addestramento e la strategia di *Ensemble*.
- Il **Capitolo 4** descriverà il motore di simulazione *Monte Carlo*, illustrando come le probabilità del singolo match vengano aggregate per simulare l'intero torneo.
- Il **Capitolo 5** affronterà l'implementazione software, mostrando come il modello sia stato integrato in un'applicazione web interattiva.
- Il **Capitolo 6** discuterà i risultati ottenuti, analizzando in modo critico le probabilità di vittoria stimate dal simulatore globale e i limiti dell'approccio statistico adottato.
- Infine, il **Capitolo 7** trarrà le conclusioni dell'intero lavoro, riassumendo gli obiettivi raggiunti e proponendo i possibili sviluppi futuri per il miglioramento dell'architettura.

2 Analisi dei Dati e Feature Engineering

2.1 I Dataset di Partenza

Per addestrare il modello e costruire il simulatore, sono stati reperiti dalla piattaforma **Kaggle** due dataset storici che coprono l'intero arco del calcio internazionale maschile. La scelta di queste fonti è ricaduta sulla loro estensione temporale (dal 1900 ad oggi) e sulla ricchezza delle informazioni, fondamentali per tracciare l'evoluzione storica della forza delle squadre e calcolare indici dinamici come il punteggio Elo.

1. Storico dei Risultati (results.csv)

Questo dataset costituisce la *ground truth* del progetto, ovvero la base su cui il modello di Machine Learning impara a riconoscere gli esiti. Contiene oltre 40.000 partite ufficiali e amichevoli disputate dalle nazionali maschili. Le colonne principali includono:

- **date**: La data esatta dell'incontro (formato AAAA-MM-GG).
- **home_team** e **away_team**: Le nazionali coinvolte.
- **home_score** e **away_score**: Il risultato finale, utilizzato per derivare la variabile target (Vittoria, Pareggio, Sconfitta).

- **tournament:** Il tipo di competizione (es. “FIFA World Cup”, “Friendly”, “UEFA Euro qualification”).
- **city, country e neutral:** La sede del match e un flag booleano che indica se la partita è stata giocata in campo neutro.

Questo dataset è fondamentale non solo per il target, ma anche per il calcolo delle feature di “stato di forma” (in questo caso la media gol segnati nelle ultime 3 partite), calcolata facendo uno *scrolling* cronologico dei punteggi.

2. Storico dei Punteggi Elo (`elo_ratings_wc2026.csv`)

Il secondo dataset è di natura puramente statistica. Contiene gli *snapshot* annuali (aggiornati al 31 dicembre di ogni anno, dal 1872 al 2026) dei punteggi Elo di ogni nazionale, adattando l’algoritmo scacchistico di Arpad Elo al calcio. Le variabili al suo interno includono:

- **snapshot_date:** La data di aggiornamento della classifica (fine anno solare).
- **country e rating:** Il nome della nazionale e il suo punteggio Elo assoluto in quella data.
- **rank:** La posizione della nazionale nella classifica Elo globale.
- **goals_for e goals_against:** Il computo storico totale delle reti segnate e subite.
- **confederation:** La federazione di appartenenza (es. UEFA, CONMEBOL).

Il **rating** Elo rappresenta la feature predittiva più potente del modello, in quanto sintetizza la forza storica e recente di una squadra in un singolo valore numerico continuo (per un approfondimento sulla formulazione matematica e sugli adattamenti specifici adottati in questo progetto, si rimanda all’Appendice A).

2.2 Il Problema del Data Leakage e l’Allineamento Temporale

L’utilizzo congiunto di questi due dataset ha presentato fin da subito una criticità strutturale che ha richiesto un intervento mirato. Il dataset dei risultati ha frequenza giornaliera, mentre nel secondo dataset l’Elo ha frequenza annuale (snapshot fissi al 31 dicembre).

Un approccio di *merge* standard (es. `pd.merge` su data e squadra) tra i due dataframe avrebbe causato *data leakage*: per una partita giocata a marzo 2023, il sistema avrebbe pescato il punteggio Elo di dicembre 2023, “barando” con informazioni future. In un modello probabilistico, questo avrebbe artificialmente gonfiato le metriche di accuratezza in fase di validazione, rendendo il modello inutile nel mondo reale.

Per risolvere questo problema, è stato necessario rispettare la *filtrazione* del processo stocastico (concetto cardine del corso di Econometria e Teoria del Portafoglio). Essa definisce l’insieme di informazioni $\mathcal{F}_t$ note fino al tempo t , garantendo che le previsioni si basino esclusivamente su dati passati ed evitando il *look-ahead bias*.

A tal fine, è stato implementato un allineamento temporale asincrono tramite la funzione `pd.merge_asof` di Pandas, configurata con il parametro `direction='backward'`. Questo

algoritmo assegna a ogni partita il punteggio Elo dell'ultimo snapshot disponibile (es. il 31 dicembre dell'anno precedente o, per le partite di dicembre, lo snapshot dell'anno prima ancora).

È doveroso precisare che, sebbene l'approccio matematicamente ideale richiederebbe l'utilizzo del rating Elo esatto calcolato alla vigilia di ogni singolo match, la natura annuale del dataset di partenza rende tale operazione non fattibile senza un oneroso ricalcolo *match-by-match*. Pertanto, l'uso dell'ultimo snapshot annuale disponibile prima della data del match rappresenta il miglior compromesso possibile: da un lato garantisce l'integrità causale dei dati, dall'altro fornisce un'approssimazione statisticamente solida e computazionalmente sostenibile della forza intrinseca della squadra.

2.3 Preprocessing e Costruzione delle Feature Predittive

Nota Metodologica sull'Ordine delle Operazioni

Nella letteratura classica, la prassi consolidata prevede che l'Analisi Esplorativa dei Dati (EDA) preceda la fase di Feature Engineering, con l'obiettivo di scoprire pattern nascosti e guidare la creazione di nuove variabili. Tuttavia, in questo progetto, l'ordine delle operazioni è stato volontariamente invertito: la costruzione delle feature è stata eseguita prima dell'EDA.

Questa scelta non è frutto di un'impostazione casuale, ma di un approccio *domain knowledge-driven* basato sulla consultazione di progetti simili presenti sulla piattaforma Kaggle e sulla letteratura sportiva applicata al machine learning. L'analisi di modelli predittivi per il calcio internazionale ha evidenziato come tre feature (la differenza di rating Elo, la forma recente e il fattore campo) rappresentino il nucleo informativo essenziale per la previsione degli esiti.

Pertanto, in questo caso, l'EDA non ha avuto il ruolo di *scoperta* di nuove feature, ma di *validazione esplorativa* delle ipotesi formulate a priori.

Pulizia e Normalizzazione dei Dati

Prima di calcolare le feature predittive, i dataset grezzi hanno richiesto una fase di pulizia e normalizzazione per garantire l'integrità matematica delle operazioni successive.

- **Gestione dei Valori Mancanti:** Il dataset dei risultati è stato epurato di qualsiasi riga contenente valori mancanti tramite la funzione `dropna()`. Nel contesto di una serie storica, l'imputazione (es. con la media) avrebbe introdotto un bias artificiale; l'eliminazione dell'istanza è stata possibile grazie alla mole di dati a disposizione.
- **Conversione da Stringa a datetime e Ordinamento Cronologico:** Le colonne contenenti le date sono state convertite in oggetti `datetime64` di Pandas tramite la funzione `pd.to_datetime()` con specifica esplicita del formato (`format='%Y-%m-%d'`). Questa conversione è il passaggio che consente l'**ordinamento cronologico** dei dataframe. Entrambi i dataframe sono quindi stati riordinati in base alla data e re-indicizzati da zero (`reset_index(drop=True)`). Questo passaggio garantisce che le operazioni di *rolling window* (per il calcolo della media gol delle ultime 3 partite) e il merge asof rispettino la causalità temporale.

Feature Engineering

Una volta puliti e allineati temporalmente i dati, sono state costruite tre feature predittive che catturano aspetti complementari della forza relativa delle squadre:

1. Differenza di Rating Elo (`elo_diff`):

Questa è la feature più potente del modello. Per ogni partita, viene calcolata la differenza tra il rating Elo della squadra di casa e quello della squadra in trasferta:

$$\text{elo_diff} = \text{elo}_{\text{home}} - \text{elo}_{\text{away}}$$

Un valore positivo indica che la squadra di casa è favorita secondo il ranking Elo, mentre un valore negativo indica il contrario.

2. Differenza di Forma Recente (`goals_diff_last_3`):

Il rating Elo è un indicatore robusto ma “lento” nel reagire a cambiamenti improvvisi di forma. Per catturare lo *stato di forma* immediato, viene calcolata la media dei gol segnati da ciascuna squadra nelle ultime 3 partite precedenti al match.

La feature finale è la differenza tra la media gol della squadra di casa e quella della squadra in trasferta:

$$\text{goals_diff_last_3} = \text{avg_goals}_{\text{home}} - \text{avg_goals}_{\text{away}}$$

Questa feature è calcolata tramite una *rolling window* di dimensione 3, applicata separatamente per ogni squadra dopo aver riordinato cronologicamente il dataset.

3. Fattore Campo (`neutral`):

Il flag booleano originale del dataset è stato convertito in intero (0 o 1) per essere utilizzato come feature numerica.

Queste tre feature costituiscono il vettore di input $\mathbf{X} = [\text{elo_diff}, \text{goals_diff_last_3}, \text{neutral}]$ che alimenta il modello di Machine Learning. La scelta di un set di feature compatto trova le seguenti motivazioni: evita il rischio di overfitting e mantiene il modello interpretabile, pur catturando le dimensioni essenziali che determinano l'esito di una partita di calcio internazionale.

2.4 Analisi Esplorativa dei Dati (EDA)

Come anticipato nella nota metodologica, l'Analisi Esplorativa dei Dati in questo progetto non ha avuto lo scopo di *scoprire* nuove feature, ma di *validare empiricamente* le assunzioni alla base del modello e confermare la bontà delle variabili ingegnerizzate. L'EDA è stata condotta utilizzando le librerie `Matplotlib` e `Seaborn`.

Di seguito vengono discusse le cinque analisi principali effettuate sul dataset unificato (`df_merged`).

1. Distribuzione degli Esiti

Il primo grafico analizza la distribuzione della variabile target.

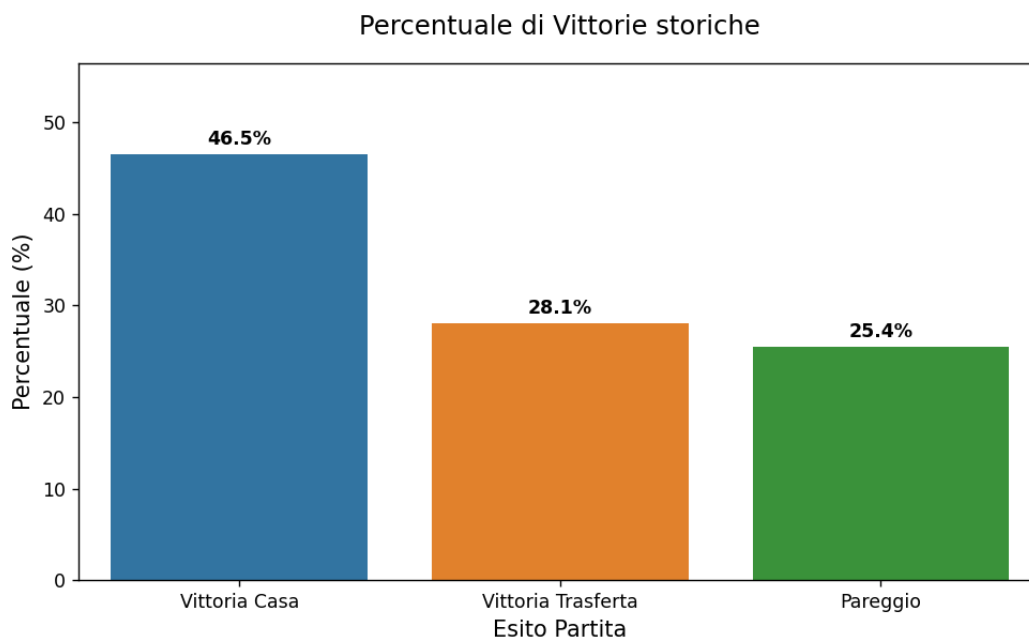


Figura 1: Distribuzione degli esiti nelle partite non neutre.

Come evidenziato dalla Figura 1, si osserva uno sbilanciamento delle classi:

- **Vittoria Casa:** 46.5% (classe maggioritaria).
- **Vittoria Trasferta:** 28.1%.
- **Pareggio:** 25.4% (classe minoritaria).

Questa asimmetria intrinseca del calcio giustifica l'adozione di strategie di bilanciamento dei pesi durante la fase di addestramento, come discusso nella Sezione 3.1.

2. Distribuzione dei Gol: Casa vs Trasferta

Per analizzare la natura *low-scoring* del calcio internazionale, è stato generato un istogramma sovrapposto che confronta la distribuzione dei gol segnati in casa e in trasferta.

La Figura 2 conferma che la maggior parte delle partite si conclude con 1 o 2 gol per squadra, con una coda lunga verso punteggi più alti. La distribuzione asimmetrica conferma il vantaggio casalingo: le squadre di casa non solo vincono più spesso, ma producono distribuzioni di gol con varianza maggiore. Questo giustifica l'uso di modelli probabilistici per la simulazione dei risultati, anziché approcci deterministici.

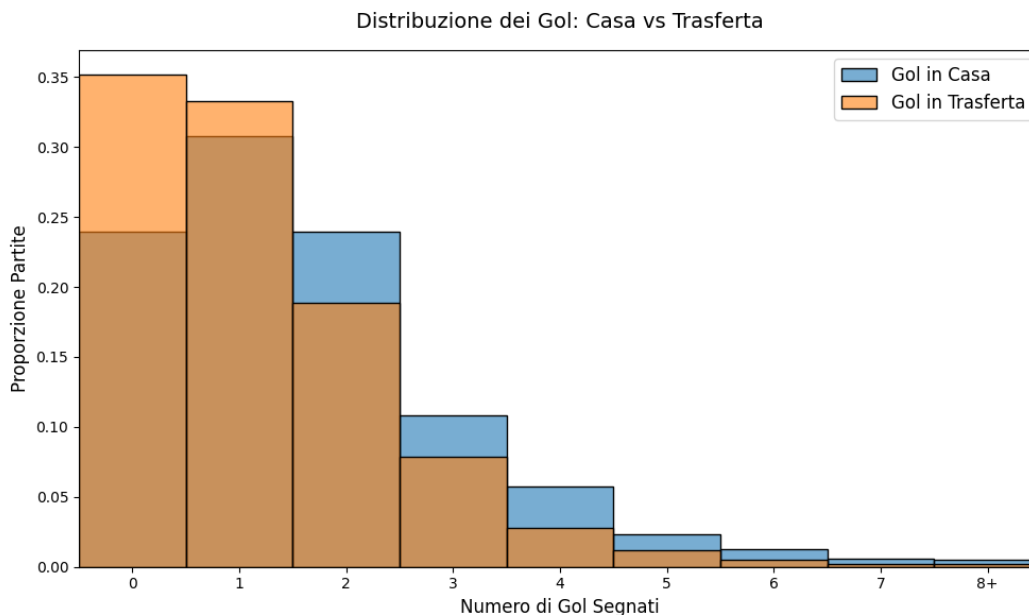


Figura 2: Distribuzione dei gol segnati.

3. Evoluzione Storica del Punteggio Elo

Per validare la robustezza del rating Elo come feature predittiva, è stato tracciato l'andamento storico delle top 7 nazionali qualificate al Mondiale attuale (Brazil, Germany, Argentina, France, Spain, England, Portugal) dal 1900 ad oggi.

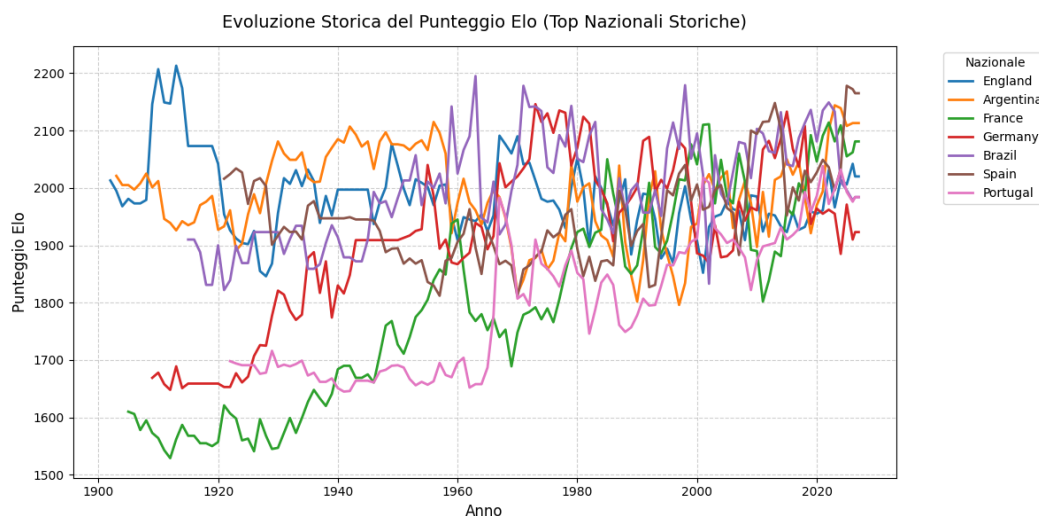


Figura 3: Evoluzione storica del rating Elo per le principali nazionali.

La Figura 3 mostra come il rating Elo cattura dinamicamente i cicli di successo delle nazionali. Le fluttuazioni riflettono eventi storici (mondiali vinti, generazioni d'oro, declini

tecnic). Questa reattività temporale è essenziale per un modello predittivo: un rating statico non potrebbe cogliere questi cambiamenti.

4. Relazione tra Gap Elo e Probabilità di Vittoria

Per verificare il potere separante della differenza Elo, è stata calcolata la probabilità di vittoria del *Team 1* (ovvero la squadra di riferimento) in funzione di bin di ΔElo .

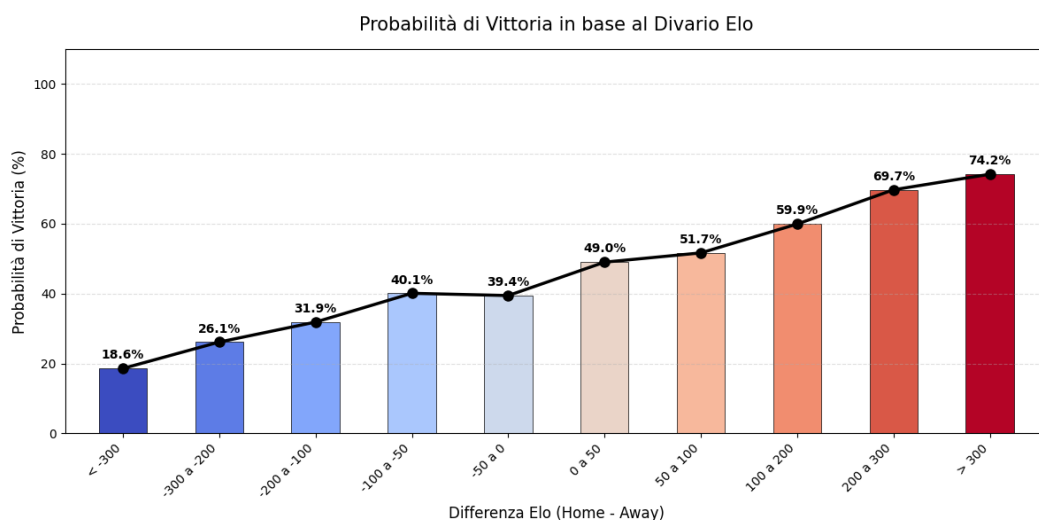


Figura 4: Probabilità di vittoria del *Team 1* in funzione della differenza Elo.

La Figura 4 evidenzia una relazione logistica: quando $\Delta\text{Elo} > 200$, la probabilità di vittoria della squadra “in casa” supera il 70%; quando $\Delta\text{Elo} < -200$, scende sotto il 30%. Questo conferma che ΔElo è una feature fortemente predittiva e correlata alla probabilità di vittoria.

5. Matrice di Correlazione tra le Variabili

Infine, è stata calcolata la matrice di correlazione di Pearson tra tutte le variabili numeriche del dataset.

La Figura 5 conferma che `elo_diff` è la feature con la correlazione più alta con il target (`home_win`), mentre `goals_diff_last_3` e `neutral` mostrano correlazioni moderate ma significative. Non si osservano correlazioni spurie tra le feature, confermando l'indipendenza delle variabili di input.

3 Architettura del Modello Predittivo

3.1 Definizione del Target e Bilanciamento delle Classi

La variabile target è stata costruita mappando la differenza di reti (`home_score - away_score`) in tre classi distinte tramite la funzione `np.select` di NumPy:

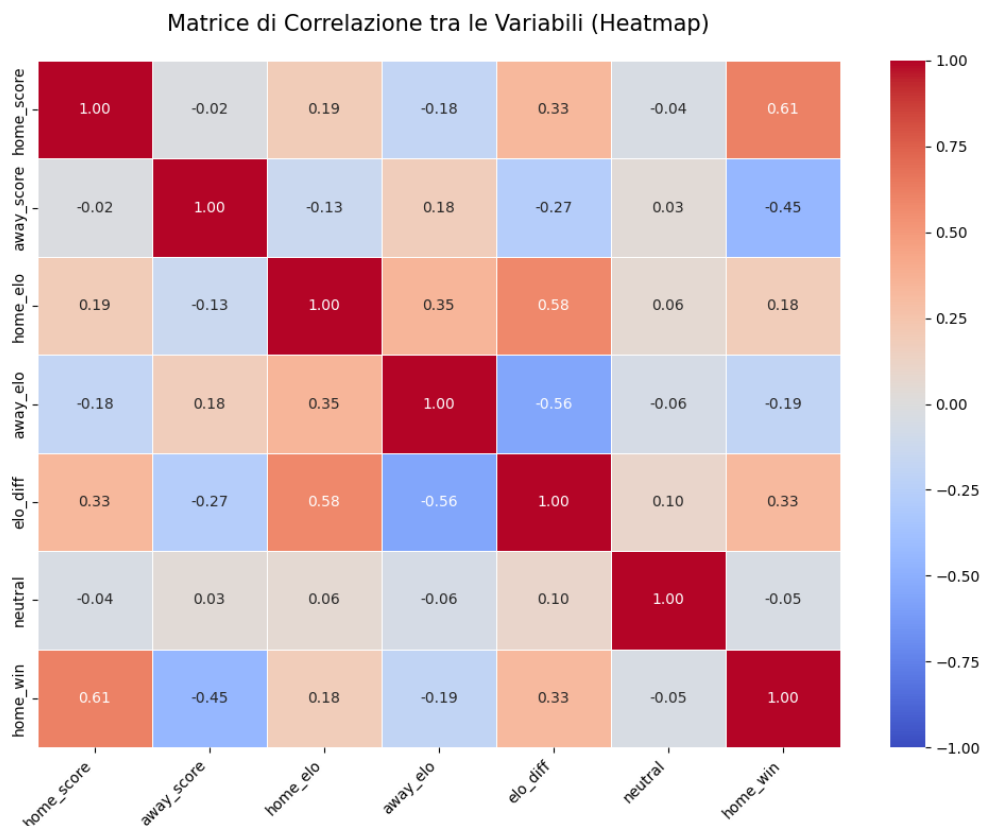


Figura 5: Heatmap di correlazione.

- **Classe 1:** Vittoria della squadra di casa.
- **Classe 0:** Pareggio.
- **Classe 2:** Vittoria della squadra in trasferta.

Come evidenziato dall'Analisi Esplorativa dei Dati (Sezione 2.4), il dataset presenta un marcato sbilanciamento delle classi, con i pareggi che costituiscono la classe minoritaria ($\approx 25\%$).

Evoluzione della Strategia di Bilanciamento In una fase iniziale dello sviluppo, la strategia di bilanciamento è stata sottovalutata: utilizzando il parametro `class_weight=None` (default), il modello ha sviluppato un comportamento di *bias* verso le classi maggioritarie. In pratica, l'algoritmo ha "imparato" che ignorare sistematicamente la classe dei pareggi e scommettere sempre su vittoria in casa o in trasferta massimizzava l'accuratezza globale, pur fallendo nel riconoscere una classe cruciale per le previsioni sportive.

Per contrastare questo fenomeno, è stato adottato il parametro `class_weight='balanced'` negli algoritmi di *Random Forest* e *LightGBM*. Questa impostazione assegna automaticamente pesi inversamente proporzionali alla frequenza delle classi nel dataset di training, secondo la formula:

$$w_j = \frac{n_{\text{samples}}}{n_{\text{classes}} \cdot n_j} \quad (1)$$

dove w_j è il peso assegnato alla classe j e n_j è il numero di campioni appartenenti a quella classe.

Questo aggiustamento della funzione di costo forza il modello a penalizzare maggiormente gli errori sulla classe minoritaria (pareggi), migliorando la capacità predittiva complessiva.

Nota implementativa: mentre Random Forest e LightGBM gestiscono il bilanciamento internamente tramite il parametro `class_weight`, per XGBoost è stato necessario calcolare esplicitamente i pesi campionari tramite `compute_sample_weight` e passarli durante la fase di `fit`, garantendo coerenza matematica su tutti e tre i modelli.

3.2 Gli Algoritmi di Machine Learning

Il cuore predittivo del sistema è costituito da un ensemble di tre algoritmi basati su alberi decisionali (*tree-based models*). La scelta di questa famiglia di algoritmi risiede nella rumorosità intrinseca dei dati sportivi, nella presenza di relazioni non lineari tra le feature e nel fatto che non richiedono la normalizzazione delle variabili di input. Tali considerazioni, apprese durante il corso di *Statistical Learning* [1], guidano la preferenza verso modelli non parametrici robusti al rumore e capaci di catturare interazioni complesse senza dover assumere distribuzioni a priori sui dati.

I tre modelli sono stati scelti per la loro complementarità teorica e per il diverso modo in cui affrontano il compromesso *bias-varianza*.

1. Random Forest

Il *Random Forest* è un algoritmo di *ensemble learning* basato sul principio del **Bootstrap Aggregating (Bagging)**. L'algoritmo costruisce una "foresta" di N alberi decisionali indipendenti, ciascuno addestrato su un sottoinsieme casuale dei dati di training (campionamento con reimmissione) e, soprattutto, utilizzando un sottoinsieme casuale delle feature ad ogni split.

Oltre al bagging sui campioni, l'introduzione della casualità nella scelta delle feature serve a *decorrelare* gli alberi. Se gli alberi fossero perfettamente correlati, la varianza dell'ensemble sarebbe uguale a quella del singolo albero; riducendo la correlazione, la varianza complessiva crolla, rendendo il modello molto robusto all'overfitting, tipico dei dati storici calcistici.

La ricerca degli iperparametri è stata condotta tramite `RandomizedSearchCV` esplorando i seguenti valori:

- `n_estimators` $\in \{100, 200, 300, 400, 500\}$: il numero di alberi nella foresta. Un valore più alto riduce la varianza ma aumenta il costo computazionale.
- `max_depth` $\in \{3, 5, 7, 10\}$: la profondità massima di ogni albero. Sono stati scelti valori non troppo alti perchè profondità eccessive, ad esempio 12, genererebbero fino a $2^{12} = 4096$ foglie, finendo per memorizzare il rumore statistico anziché apprendere regole generali.

- `min_samples_split` $\in \{2, 5, 10\}$ e `min_samples_leaf` $\in \{1, 2, 4\}$: parametri di regolarizzazione che impediscono la creazione di nodi e foglie su sottoinsiemi troppo piccoli, contenendo la sensibilità al rumore locale.
- `max_features` $\in \{1, 2, 3\}$: il numero di feature considerate ad ogni split. I classici valori `sqrt` e `log2`, su un feature space di soli 3 elementi, restituirebbero $\sqrt{3} \approx 1.73$ e $\log_2(3) \approx 1.58$, che scikit-learn arrotonda per difetto a 1. Sebbene costringere gli alberi a valutare una sola feature per split massimizzi la decorrelazione, rischia di renderli individualmente troppo deboli (underfitting), poiché le divisioni avverrebbero essenzialmente "alla cieca". Esplorare esplicitamente $\{1, 2, 3\}$ permette al modello di trovare il compromesso ideale tra la diversità degli alberi (valori bassi) e la loro forza predittiva individuale (valori alti).

2. XGBoost (Extreme Gradient Boosting)

A differenza del Random Forest, **XGBoost** è un algoritmo di **Gradient Boosting** sequenziale. Invece di costruire alberi indipendenti, XGBoost li costruisce in serie: ogni nuovo albero $h_m(x)$ viene addestrato per correggere gli errori residui commessi dal modello cumulativo precedente $F_{m-1}(x)$:

$$F_m(x) = F_{m-1}(x) + \gamma \cdot h_m(x)$$

dove γ è il *learning rate*.

Poiché il boosting sequenziale è strutturalmente propenso all'overfitting, la griglia di ricerca è stata focalizzata sul controllo della stocasticità e della complessità:

- `n_estimators` $\in \{100, 200, 300, 400, 500\}$ e `learning_rate` $\in \{0.01, 0.05, 0.1\}$: il compromesso tra numero di alberi e fattore di shrinkage. Un learning rate basso richiede più iterazioni per convergere, ma produce un modello più stabile.
- `max_depth` $\in \{2, 3, 5, 7\}$: la profondità massima degli alberi. Il limite inferiore rispetto a Random Forest (tetto a 7) riflette la maggiore aggressività del boosting: alberi troppo profondi in un contesto sequenziale amplificano il rischio di overfitting ad ogni iterazione.
- `subsample` $\in \{0.6, 0.8, 1.0\}$: la frazione di righe del dataset campionate (senza reimmissione) per addestrare ogni singolo albero.
- `colsample_bytree` $\in \{0.66, 1.0\}$: la frazione di feature campionate per ogni albero. Il valore 0.66 è stato scelto volontariamente: moltiplicato per le 3 feature disponibili, restituisce esattamente 2, assicurando varietà tra gli stimatori.
- `min_child_weight` $\in \{1, 5, 10, 20\}$: la somma minima dei pesi dell'Hessiana (derivata seconda della funzione di costo) richiesta in un nodo figlio affinché lo split sia accettato. Questo parametro introduce una regolarizzazione: penalizza i tentativi del modello di effettuare split su sottoinsiemi in cui la curvatura della loss è troppo bassa, bloccando la memorizzazione di rumore locale e outlier.

3. LightGBM (Light Gradient Boosting Machine)

LightGBM è una variante altamente ottimizzata del gradient boosting. Pur condividendo la filosofia sequenziale di XGBoost, introduce due innovazioni algoritmiche fondamentali: l'*Histogram-based Learning* (discretizzazione delle feature in bin) e il *Leaf-wise (Best-first) Growth* (crescita dell'albero scegliendo la foglia con la massima riduzione della loss).

Il meccanismo *leaf-wise* è estremamente potente ma aggressivo: tende a creare alberi molto profondi e asimmetrici, aumentando il rischio di overfitting su dataset piccoli. Per questo motivo, i parametri di regolarizzazione esplorati sono stati calibrati per “frenare” questa crescita:

- `n_estimators` $\in \{100, 200, 300, 400, 500\}$ e `learning_rate` $\in \{0.01, 0.05, 0.1\}$: identici a XGBoost, per mantenere comparabilità tra i due modelli di boosting.
- `num_leaves` $\in \{7, 15, 25\}$: il numero massimo di foglie. In LightGBM questo è il parametro di controllo della complessità più importante, sostituendo il concetto di profondità massima. Poiché l'albero cresce *leaf-wise*, limitare il numero di foglie impedisce che un singolo albero diventi troppo complesso.
- `min_child_samples` $\in \{10, 20, 50, 100\}$: il numero minimo di campioni richiesti in una foglia.

L'utilizzo congiunto di questi tre modelli crea un ecosistema predittivo robusto: il Random Forest fornisce una baseline stabile e a bassa varianza, mentre XGBoost e LightGBM spingono al massimo la capacità di apprendimento dei pattern complessi, compensando i rispettivi punti deboli attraverso la successiva fase di Soft Ensemble.

Sintesi del Tuning

I migliori iperparametri trovati per ciascun modello sono:

- **Random Forest:** `max_depth=3`, `max_features=3`, `min_samples_split=10`, `min_samples_leaf=1`, `n_estimators=200`.
- **XGBoost:** `max_depth=2`, `learning_rate=0.01`, `n_estimators=300`, `subsample=0.6`, `colsample_bytree=1.0`, `min_child_weight=10`.
- **LightGBM:** `num_leaves=7`, `learning_rate=0.01`, `n_estimators=200`, `min_child_samples=20`.

3.3 Strategia di Addestramento

L'addestramento dei modelli è stato gestito in modo da preservare l'integrità temporale dei dati.

Time Series Cross-Validation

In contesti sportivi e finanziari, una classica k -fold cross-validation è matematicamente scorretta poiché mescolerebbe dati passati e futuri, violando il principio di causalità e introducendo *data leakage*. Per evitare questo, è stata implementata la `TimeSeriesSplit` di `scikit-learn`, configurata con `n_splits=3`.

Questo metodo genera split temporali consecutivi: il modello viene addestrato su una finestra temporale crescente e validato sul blocco temporale immediatamente successivo. In questo modo, la valutazione simula le condizioni reali di previsione, dove il modello deve generalizzare su dati cronologicamente consecutivi ma mai visti durante la fase di training.

Ottimizzazione degli Iperparametri e Metrica di Scoring

La ricerca degli iperparametri è stata condotta tramite `RandomizedSearchCV`, impostato con `n_iter=50` per garantire un'esplorazione più capillare e approfondita dello spazio dei parametri.

Un'ulteriore accortezza implementativa ha riguardato l'ottimizzazione computazionale. Sia nei classificatori di boosting (XGBoost e LightGBM) che nella `RandomizedSearchCV`, è stato impostato il parametro `n_jobs=-1`. Questa istruzione forza le librerie a sfruttare tutti i core logici della CPU disponibili, parallelizzando la costruzione degli alberi e le iterazioni della cross-validation, in modo da ridurre la velocità di addestramento.

La scelta cruciale è ricaduta sulla metrica di ottimizzazione: non l'accuratezza, ma la **Log Loss negativa** (`scoring='neg_log_loss'`). A differenza dell'accuratezza, che valuta solo se la classe predetta è corretta (trattando una previsione al 51% e una al 99% allo stesso modo), la Log Loss è una *strictly proper scoring rule*. Essa penalizza severamente le previsioni espresse con alta confidenza che si rivelano errate. Ottimizzare la Log Loss forza il modello a restituire probabilità ben calibrate (es. un evento previsto al 70% deve verificarsi empiricamente circa il 70% delle volte).

Holdout Set Temporale

Per la valutazione finale, il dataset è stato partizionato con un taglio netto alla data del 2024-01-01 (7020 partite). Tutte le partite precedenti sono state usate per il training e la **cross-validation**, mentre quelle dal 2024 in poi hanno costituito l'*holdout set* (350 partite). Questo garantisce che la metrica finale misuri la reale capacità predittiva del modello su un periodo storico inedito, simulando il comportamento del modello alla vigilia del Mondiale 2026.

3.4 Il Soft Ensemble

Per combinare le uscite dei tre modelli (Random Forest, XGBoost, LightGBM), si è evitato il classico *hard voting* (la regola della maggioranza), che è un'operazione che scarta informazioni preziose sulle incertezze marginali. È stata invece implementata una **Soft Ensemble** basata sulla media aritmetica dei vettori di probabilità:

$$P_{\text{ensemble}}(y \mid \mathbf{x}) = \frac{1}{3} \sum_{m=1}^3 P_m(y \mid \mathbf{x})$$

dove $P_m(y \mid \mathbf{x})$ è la distribuzione di probabilità restituita dal modello m -esimo per le tre classi $y \in \{1, 0, 2\}$. Questa tecnica di *probability averaging* “smussa” le previsioni estreme o troppo confidenti dei singoli algoritmi, riducendo la varianza della previsione finale.

Giustificazione dell’Ensemble e Impatto sulla Simulazione

Alla luce delle analisi di performance, l’Ensemble non emerge come il vincitore assoluto su nessuna singola metrica, ma si conferma il **compromesso ottimale** per lo scopo del progetto. Mentre i singoli algoritmi mostrano punti di forza specifici (es. la calibrazione di Random Forest o l’accuratezza di LightGBM), l’Ensemble eccelle nella **stabilità complessiva**.

In un contesto di simulazione Monte Carlo, dove l’obiettivo è generare migliaia di scenari realistici piuttosto che indovinare il singolo esito, la robustezza della distribuzione di probabilità in ingresso è fondamentale. Un modello che eccelle in Accuracy ma soffre di calibrazione produrrebbe scenari distorti, sovrastimando la probabilità di eventi rari o sottostimando quelli frequenti. L’Ensemble, combinando le forze di tre algoritmi complementari, mitiga questi rischi, fornendo al motore stocastico del simulatore probabilità affidabili e prive di bias sistematici.

3.5 Valutazione delle Performance

Le prestazioni dei modelli sono state valutate sull’*holdout set* temporale, composto da 350 partite giocate dal 2024 in poi. La tabella seguente riassume le metriche principali:

Modello	Log Loss	Accuracy	Macro Precision	Tempo (s)
Random Forest	1.0210	0.4486	0.4150	10.30
XGBoost	1.0308	0.4629	0.4279	4.35
LightGBM	1.0329	0.4771	0.4286	22.43
Ensemble	1.0273	0.4543	0.4129	-

Tabella 1: Confronto delle performance sull’holdout set temporale.

Analisi della Calibrazione Probabilistica

La metrica primaria in questo caso non è l’accuratezza, ma la **Log Loss**, poiché il modello verrà inserito in un simulatore Monte Carlo dove la calibrazione delle probabilità è cruciale (*Garbage-In, Garbage-Out*). In questo senso, **Random Forest** ottiene il risultato migliore (1.0210), seguito dall’Ensemble (1.0273). Questo indica che RF produce le probabilità

¹Per le metriche di valutazione, si distinguono due categorie: “lower is better” per Log Loss e Tempo di addestramento (valori inferiori indicano prestazioni migliori) e “higher is better” per Accuracy e Macro Precision (valori superiori indicano prestazioni migliori).

meglio calibrate: quando assegna una probabilità del 70% a un evento, questo si verifica empiricamente circa il 70% delle volte.

L'Ensemble, pur non raggiungendo la calibrazione di RF, migliora significativamente quella di XGBoost (1.0309) e LightGBM (1.0329), confermando l'efficacia della tecnica di *probability averaging* nel ridurre la varianza e le previsioni troppo confidenti.

Analisi dell'Accuratezza e della Macro Precision

Sebbene la Log Loss sia la metrica dominante per questo progetto, l'analisi dell'**Accuracy** e della **Macro Precision** fornisce informazioni complementari sul comportamento dei modelli:

- **Accuracy:** LightGBM ottiene il risultato migliore (0.4771), seguito da XGBoost ed Ensemble (entrambi 0.4571), mentre Random Forest si colloca in ultima posizione (0.4486). Questo risultato potrebbe suggerire che LightGBM sia il modello “più bravo” a indovinare l'esito secco, ma è un'interpretazione fuorviante: un modello può essere molto accurato ma mal calibrato (es. prevedere sempre la classe maggioritaria con alta confidenza), rendendolo inutile per la simulazione stocastica.
- **Macro Precision:** Anche qui LightGBM vince (0.4286), seguito da XGBoost (0.4270), Ensemble (0.4180) e Random Forest (0.4150). La Macro Precision è particolarmente informativa perché tratta tutte le classi con pari importanza, evitando che il modello sia premiato solo per la sua capacità di prevedere la classe maggioritaria. Il fatto che LightGBM ottenga sia l'Accuracy più alta che la Macro Precision più alta suggerisce che ha imparato a riconoscere meglio anche i pareggi, ma a scapito di una calibrazione probabilistica leggermente peggiore.

L'Ensemble si posiziona in una zona intermedia: non è il più accurato, ma mantiene un'Accuracy competitiva (0.4571) e una Macro Precision discreta (0.4180).

Analisi del Tempo di Addestramento e Robustezza Hardware

Un ultimo aspetto pratico rilevante è il **tempo di addestramento**. I tempi riportati nella Tabella 1 si riferiscono all'esecuzione su un PC desktop dotato di CPU AMD Ryzen 7 9700X (8-Core):

- **XGBoost** è nettamente il più efficiente (4.35 secondi), grazie alla sua implementazione ottimizzata in C++ e alla parallelizzazione nativa, che sfrutta appieno gli 8-Core del processore.
- **Random Forest** (10.30 secondi) si posiziona in una via di mezzo, con un tempo circa 2.4 volte superiore a XGBoost. Questo è dovuto al fatto che costruisce 500 alberi indipendenti in parallelo, ma ciascun albero è relativamente semplice (profondità massima limitata).
- **LightGBM** (22.43 secondi) risulta essere il più lento, circa 5 volte superiore a XGBoost. Questo risultato è controintuitivo, dato che LightGBM è noto per la sua efficienza. La spiegazione risiede nel meccanismo *leaf-wise growth*: a differenza della crescita

per livello, la selezione della foglia con la massima riduzione della loss richiede calcoli più complessi e non parallelizzabili nello stesso modo, specialmente con i parametri ottimizzati (`num_leaves` fino a 25, `min_child_samples` fino a 100) che creano alberi asimmetrici e profondi.

L'Ensemble, combinando tutti e tre, richiede il tempo cumulativo dei tre addestramenti (circa 37 secondi totali), ma questo è un costo accettabile per un sistema che viene eseguito una sola volta in fase di sviluppo e poi utilizzato per inferenza rapida.

Per verificare la stabilità del modello rispetto all'hardware sottostante, l'addestramento è stato replicato su un PC portatile dotato di un processore 11th Gen Intel Core i5-1135G7. I risultati ottenuti sono riassunti nella tabella seguente:

Configurazione	Modello	Log Loss	Accuracy	Tempo (s)
Desktop (Ryzen 7 9700X)	Random Forest	1.0210	0.4486	10.30
	XGBoost	1.0308	0.4629	4.35
	LightGBM	1.0329	0.4771	22.43
Portatile (Intel Core i5-1135G7)	Random Forest	1.0210	0.4486	79.15
	XGBoost	1.0309	0.4571	22.18
	LightGBM	1.0329	0.4771	80.46

Tabella 2: Confronto delle performance tra desktop e portatile.

Come evidenziato dalla Tabella 2, le metriche predittive (Log Loss e Accuracy) sono **praticamente identiche** tra le due configurazioni, con differenze inferiori allo 0.01%. Questo conferma che il modello è *riproducibile*: le performance non dipendono dall'architettura hardware, ma esclusivamente dai dati e dagli iperparametri ottimizzati.

L'unica differenza significativa riguarda i **tempi di addestramento**: il desktop riduce i tempi di un fattore 7-8x.

Questa robustezza hardware è un requisito fondamentale per il deploy dell'applicazione Streamlit: il modello può essere addestrato una sola volta su una macchina potente, salvato tramite `joblib`, e poi caricato rapidamente su qualsiasi dispositivo per l'inferenza in tempo reale.

Analisi per Classe e il Problema dei Pareggi

Il *classification report* fornisce una valutazione delle performance per ciascuna classe c , analizzando tre metriche fondamentali:

- La **Precision**, analiticamente definita come $P_c = \frac{TP_c}{TP_c + FP_c}$ (ovvero il rapporto tra i veri positivi (TP_c) e il totale delle istanze previste come positive, includendo i falsi positivi (FP_c)), misura l'affidabilità della previsione: quando il modello prevede un certo esito, quante volte ha effettivamente ragione?
- Il **Recall**, analiticamente definito come $R_c = \frac{TP_c}{TP_c + FN_c}$ (ovvero il rapporto tra i veri positivi e il totale degli eventi realmente accaduti per quella classe, includendo i falsi

negativi (FN_c)), indica la capacità di intercettazione: su tutti gli eventi di classe c , quanti il modello è riuscito a individuarne?

- **L’F1-Score**, analiticamente definito come $F1_c = 2 \cdot \frac{P_c \cdot R_c}{P_c + R_c}$ (ovvero la media armonica tra le due metriche), è un indicatore di sintesi cruciale.

L’F1-Score bilancia la qualità e la quantità delle previsioni: a differenza della media aritmetica, la media armonica penalizza fortemente i modelli che eccellono in una metrica ma crollano nell’altra, risultando particolarmente informativa in dataset sbilanciati dove le classi minoritarie (come i pareggi) sono intrinsecamente più difficili da prevedere.

La tabella seguente riassume i valori di queste metriche per ciascun modello sull’holdout set:

Modello	Classe	Precision	Recall	F1-Score
Random Forest	0 (Pareggio)	0.22	0.18	0.20
	1 (Casa)	0.56	0.61	0.58
	2 (Trasferta)	0.46	0.49	0.48
XGBoost	0 (Pareggio)	0.24	0.19	0.21
	1 (Casa)	0.57	0.63	0.60
	2 (Trasferta)	0.48	0.51	0.49
LightGBM	0 (Pareggio)	0.26	0.15	0.19
	1 (Casa)	0.56	0.67	0.61
	2 (Trasferta)	0.46	0.54	0.50
Ensemble	0 (Pareggio)	0.22	0.16	0.19
	1 (Casa)	0.56	0.62	0.59
	2 (Trasferta)	0.45	0.52	0.48

Tabella 3: Metriche per classe per ciascun modello sull’holdout set.

Dall’analisi della Tabella 3 emergono tre pattern distinti:

- **Classe 1 (Vittoria Casa):** Tutti i modelli performano bene, con precision tra 0.56-0.57 e recall tra 0.61-0.67. LightGBM ottiene il recall più alto (0.67), indicando che riesce a identificare quasi i due terzi delle vittorie casalinghe reali.
- **Classe 2 (Vittoria Trasferta):** Performance intermedia, con precision 0.45-0.48 e recall 0.49-0.54. XGBoost mostra la precision più alta (0.48), mentre LightGBM ottiene il recall migliore (0.54).
- **Classe 0 (Pareggio):** Qui risiede la criticità. Nonostante il bilanciamento delle classi (`class_weight='balanced'`), il recall rimane estremamente basso (0.15-0.19), indicando che il modello identifica correttamente solo il 15-19% dei pareggi reali. La precision è leggermente migliore (0.22-0.26), ma comunque modesta. LightGBM, pur ottenendo la precision più alta (0.26), ha il recall più basso (0.15), suggerendo che il modello è molto conservativo: quando prevede un pareggio, spesso ha ragione, ma ne “perde” l’85%.

Questo risultato riflette la natura stocastica del calcio: i pareggi sono eventi a bassa probabilità, spesso determinati da fattori non catturabili dalle feature disponibili.

4 Il Motore di Simulazione Monte Carlo

Completato il modello predittivo, il passo successivo è la costruzione del simulatore. Poiché l'obiettivo è proiettare l'intero andamento della competizione, il sistema non si limita a calcolare le probabilità di un singolo incontro in modo statico. Il programma genera migliaia di scenari possibili partendo dalle statistiche pre-torneo, simulando ogni fase e aggiornando dinamicamente la forza delle squadre (il punteggio Elo) all'interno di ogni singola iterazione, per riflettere i picchi o i crolli di forma che caratterizzano un Mondiale reale.

4.1 Caricamento delle Risorse e Struttura Algoritmica

Prima di innescare la logica del simulatore, il sistema gestisce il recupero degli **oggetti predittivi**. I tre algoritmi di *Machine Learning* costituenti l'Ensemble vengono de-serializzati dal disco, recuperando i pesi e gli iperparametri precedentemente ottimizzati nella fase di *training*.

Contestualmente ai modelli, viene importato il dataset contenente le basi statistiche delle formazioni. Per garantire l'efficienza computazionale durante l'esecuzione del simulatore, l'acquisizione e l'allocazione di queste risorse in memoria avviene una singola volta al primo avvio, abbattendo drasticamente i tempi di latenza e fornendo le basi per le migliaia di inferenze successive.

Completata questa operazione preliminare, l'architettura del motore viene organizzata attraverso una **pipeline di funzioni modulari**. Queste funzioni gestiscono in ordine logico tutte le fasi del torneo:

- Calcolo delle probabilità e risoluzione stocastica della **singola partita**.
- Regola matematica per l'**aggiornamento dinamico** del punteggio Elo post-match.
- Iterazione e stesura delle classifiche per i **gironi iniziali**.
- Risoluzione del **tabellone a eliminazione diretta**.

4.2 Simulazione del Singolo Match e Varianza Stocastica

La funzione `simulate_match_ensemble` gestisce la logica algoritmica del singolo scontro. L'operazione iniziale consiste nella determinazione del **vantaggio del campo**. L'algoritmo verifica se una delle due formazioni figura nella lista delle nazioni ospitanti (`host_nations`). In caso affermativo, la partita non viene considerata neutrale (`is_neutral = 0`) e, qualora la squadra ospitante figuri formalmente in trasferta, le variabili vengono invertite (`scambio_effettuato = True`) per garantire che il modello valuti correttamente l'impatto del fattore casalingo.

Il passaggio critico della simulazione è l'introduzione della **stocasticità**. Se il modello predittivo ricevesse in input i punteggi Elo statici, le probabilità stimate per un medesimo

scontro risulterebbero identiche in ogni iterazione, vanificando l'utilità del metodo Monte Carlo. Per modellare le fluttuazioni di forma e l'imprevedibilità tipica del calcio, l'Elo di base di ciascuna squadra viene perturbato. Ciò avviene campionando un valore da una distribuzione normale tramite la funzione `np.random.normal(loc=elo_base, scale=75)`.

La taratura di questo iperparametro è stata supportata da una consultazione teorica con un *Large Language Model* (Qwen 3.7-Plus), interrogato mediante la seguente istruzione di *prompt engineering*:

“Agisci come un Senior Data Scientist esperto in modellazione predittiva sportiva e simulazioni Monte Carlo per il calcio. Nel mio algoritmo, utilizzo il punteggio Elo come base per prevedere l'esito dei match. Per simulare la varianza di forma e l'imprevedibilità a breve termine, applico una perturbazione stocastica al punteggio Elo base di ogni squadra campionando da una distribuzione normale gaussiana. Quale dovrebbe essere un valore ottimale o un range consigliato per l'iperparametro deviazione standard (σ)? Fornisci una giustificazione matematica e probabilistica basata sulla scala standard dei rating Elo e sui tassi di upset realistici.”

L'analisi probabilistica generata ha confermato che $\sigma = 75$ rappresenta un punto di ottimo teorico per calibrare l'aggiunta di **varianza gaussiana**. Innanzitutto, tale deviazione è coerente con l'ordine di grandezza del vantaggio casalingo nel calcio (stimato storicamente tra i 60 e i 100 punti Elo). Inoltre, perturbando entrambe le formazioni in modo indipendente, la deviazione standard della differenza combinata risulta pari a:

$$\sigma_{diff} = \sqrt{75^2 + 75^2} \approx 106$$

Dal punto di vista statistico, ciò garantisce tassi di *upset* (risultati a sorpresa) perfettamente allineati alla realtà del calcio internazionale: a fronte di un divario teorico di 100 punti Elo, il rumore ribalta i pronostici in circa il 16% delle simulazioni (che scende al 3% per un divario di 200 punti). Questo bilanciamento preserva l'informatività del rating di base, innescando però l'imprevedibilità intrinseca dello sport.

Calcolate le *feature* predittive (`elo_diff` perturbato e `goals_diff`), i dati alimentano il *Soft Ensemble* per generare il vettore delle probabilità in uscita. Qualora il match in esame appartenga a un turno a eliminazione diretta (`is_knockout = True`), il pareggio diviene un esito inammissibile. In questo caso, la probabilità associata al pareggio (`target = 0`) viene matematicamente azzerata e le probabilità di vittoria delle due formazioni vengono normalizzate dividendo ciascun valore per la loro somma parziale ($P_A + P_B$).

È doveroso specificare la consapevolezza della **non ottimalità** di tale procedura: questa ripartizione proporzionale rappresenta una semplificazione. Essa assume matematicamente che la differenza di forza tra le due formazioni rimanga invariato anche in caso di tempi supplementari o calci di rigore. Nella realtà sportiva, la stocasticità aumenta e le probabilità di successo tendono asintoticamente al 50%. Tuttavia, ai fini del presente elaborato, tale normalizzazione fornisce un compromesso logicamente solido e computazionalmente efficiente per forzare un esito binario, senza incorrere nella necessità di addestrare un modello secondario dedicato esclusivamente all'estrazione dei rigori.

Infine, l'esito della partita non viene assegnato in modo deterministico alla classe con la probabilità maggiore, ma viene estratto in modo **casuale** mediante la funzione

`np.random.choice([0, 1, 2], p = probs)`. In questo modo, il risultato viene campionato ponderandolo sulla distribuzione stimata dal modello: un evento accreditato del 20% di probabilità si verificherà in circa un quinto degli scenari esplorati, garantendo la corretta propagazione dell'incertezza per tutto lo sviluppo del torneo.

4.3 Aggiornamento Dinamico dell'Elo

Affinché il simulatore replichi fedelmente le dinamiche tecniche e psicologiche di un torneo (il cosiddetto *momentum*), i rating non possono rimanere ancorati ai valori dello “*stato zero*”. Al termine di ogni singolo incontro simulato, la funzione `calcola_nuovo_elo_sim` aggiorna dinamicamente la forza relativa delle squadre in base all'esito ottenuto dal modello.

L'algoritmo confronta l'aspettativa matematica pre-match con l'esito effettivamente verificatosi. Come per la stima della varianza stocastica, la calibrazione del moltiplicatore *K-Factor*, che funge matematicamente da *learning rate* del sistema, è stata supportata da una consultazione mirata con il LLM Qwen 3.7-Plus, interrogato mediante il seguente *prompt*:

*“Agisci come un Senior Data Scientist esperto in modelli predittivi per il calcio. In una bozza del mio codice Python per l'aggiornamento dell'Elo post-partita, ho impostato il parametro in questo modo: $K = \{\}$ if *is_knockout* else $\{\}$. Quali valori scalari ottimali dovrei inserire per i gironi e per le fasi a eliminazione diretta? Qual è la regola d'oro per bilanciare questo parametro post-match con la deviazione standard pre-match ($\sigma = 75$) che uso per perturbare l'Elo? Fornisci una giustificazione analitica.”*

L'analisi ha raccomandato l'adozione dei valori $K = 30$ per la fase a gironi e $K = 45$ per i turni a eliminazione diretta.

Da un punto di vista analitico, un $K = 30$ implica che il massimo scostamento possibile in una singola gara sia di ± 30 punti Elo. Questo valore risulta ottimale: garantisce una convergenza ai cambiamenti reali di forma nell'arco di 5-10 partite, senza far crollare il rating di una favorita per un singolo scivolone. L'innalzamento del moltiplicatore a 45 (un incremento del 50%) per le fasi *knockout* riflette invece la maggiore varianza intrinseca e l'elevato peso sportivo e informativo che tali scontri portano con sé.

Un *insight* fondamentale emerso dallo studio è la **sinergia critica tra σ e K** . Avendo precedentemente fissato una perturbazione stocastica pre-match di $\sigma = 75$, il sistema necessita di un K bilanciato per evitare l'*overreaction* post-match. Un K eccessivamente alto abbinato a un σ alto costringerebbe il modello a inseguire il proprio rumore, distruggendo l'informatività del rating di base.

Completato l'aggiornamento numerico, i nuovi punteggi vengono sovrascritti nel dizionario operativo del torneo. Per preservare la congruenza logica con il vantaggio del campo, se all'inizio della simulazione era stata effettuata un'inversione delle formazioni per gestire una nazionale ospitante (`scambio_effettuato = True`), il sistema re-inverte i punteggi Elo aggiornati, assegnandoli correttamente alle rispettive squadre prima di procedere con il match successivo.

4.4 Logica del Tabellone a Eliminazione Diretta

Il modulo `simulate_tournament_phase` gestisce l'intero ciclo di vita di una singola iterazione Monte Carlo, traducendo in logica software il formato FIFA a 48 squadre introdotto per il Mondiale 2026.

La competizione si apre con la **fase a gironi**. Iterando sui 12 gruppi predefiniti (`gironi_ufficiali`), la funzione `simulate_group` sfrutta l'oggetto `combinations` del modulo nativo `itertools` per generare il calendario degli scontri (6 incontri per ciascun raggruppamento). Terminata la simulazione dei match, il sistema accumula i punti secondo le regole standard: 3 punti per la vittoria, 1 per il pareggio e 0 per la sconfitta.

A questo punto emerge una limitazione del *framework* predittivo: essendo il modello addestrato unicamente a stimare probabilità di esito categoriale (1, X, 2), la generazione degli *scoreline* esatti risulta analiticamente incalcolabile. Di conseguenza, il criterio della differenza reti risulta inapplicabile. Per ovviare al problema, l'algoritmo adotta come **criterio di spareggio** il punteggio Elo corrente al momento della chiusura del girone. Si assume che, a parità di punti cumulati, a meritare la qualificazione sia la squadra che dimostra una forza intrinseca (storica e di forma) superiore.

Chiusa la fase a gironi, il sistema qualifica le prime due classificate di ogni gruppo (24 nazionali). Parallelamente, i dati delle formazioni giunte al terzo posto vengono aggregati in una lista e riordinati per punti e punteggio Elo, permettendo il **ripescaggio** delle migliori 8 formazioni, completando così il *roster* delle 32 qualificate.

Il passaggio successivo consiste nell'operazione di **costruzione del tabellone** per i sedicesimi di finale. L'algoritmo implementa una logica di accoppiamento iterativa volta a minimizzare, per quanto matematicamente possibile, l'eventualità di uno scontro tra formazioni provenienti dal medesimo girone:

1. Le 8 formazioni ripescate (terze classificate) vengono accoppiate sequenzialmente con 8 delle formazioni arrivate prime, vincolando l'assegnazione alla condizione `girone_p != girone_t`.
2. Le 4 prime classificate rimanenti vengono incrociate con 4 delle seconde classificate (`girone_p != girone_s`).
3. Le ultime 8 seconde classificate vengono incrociate tra di loro. In questo caso, il sistema tenta di rispettare il vincolo sui gironi d'origine; tuttavia, qualora le combinazioni residue non lo permettano, entra in gioco un *fallback* logico che forza l'accoppiamento tra le prime due squadre disponibili, onde evitare stalli computazionali.

Risolti gli accoppiamenti, il torneo evolve esclusivamente a eliminazioni dirette. La funzione innestata `gioca_turno_semplice` accetta la lista delle squadre ancora in gara, le processa a due a due e invoca il simulatore di match con il parametro `is.knockout = True`. Questa chiamata innesca automaticamente l'azzeramento della probabilità di pareggio e l'innalzamento del moltiplicatore dell'Elo ($K = 45$), come discusso nelle sezioni precedenti.

In modo **ricorsivo**, `gioca_turno_semplice` riceve l'output del turno precedente (dimezzando progressivamente l'insieme delle squadre) processando in sequenza sedicesimi, ottavi, quarti e semifinali. Il *loop* si conclude con la simulazione della finale, restituendo come output

dell'intero processo la singola nazione campionessa, che andrà a incrementare il proprio contatore delle vittorie nelle statistiche finali del Monte Carlo.

5 Implementazione Software e Deploy

Per trasformare l'architettura algoritmica e il motore stocastico in uno strumento interattivo ed esplorabile, è stata sviluppata un'applicazione web dedicata utilizzando *Streamlit*, un *framework* Python ideato specificamente per il *deploy* rapido di modelli di *Machine Learning* e *Data Science*.

L'obiettivo di questa fase è stato fornire un'interfaccia utente (UI) reattiva per l'esplorazione analitica delle probabilità.

5.1 Gestione della Configurazione e dei Dati Live

Affinché il simulatore possa essere aggiornato in tempo reale senza la necessità di alterare o ricompilare lo *script* principale (`app.py`), tutte le variabili di stato esterne sono state serializzate in un file strutturato denominato `config_mondiale.json`.

Questo file funge da base di conoscenza statica e fornisce all'applicazione quattro strutture dati fondamentali al momento dell'avvio:

- L'elenco delle nazioni ospitanti (`host_nations`), cruciale per l'assegnazione algoritmica del vantaggio casalingo.
- I raggruppamenti ufficiali del torneo (`gironi_ufficiali`), necessari per generare la fase a gruppi.
- I punteggi correnti (`latest_elo`), che vengono impiegati dal modulo di previsione del singolo match per valutare le probabilità senza l'applicazione di perturbazioni stocastiche.
- Lo storico in continuo aggiornamento delle gare reali disputate (`partite_reali`). Poiché il formato JSON non supporta nativamente l'utilizzo di tuple per le chiavi di un dizionario, si è optato per una struttura in cui i nomi delle due formazioni sono concatenati da un separatore logico (es. "Team A|Team B"), mappati sull'esito finale della gara sotto forma di lista numerica.

Oltre a fornire i dati strutturali, il sistema è progettato per assimilare i risultati reali man mano che le partite del Mondiale vengono disputate. Questo processo è fondamentale per il modulo di previsione del singolo match, che necessita di dati sempre freschi per calcolare le probabilità odierne.

Leggendo in ordine cronologico le partite già concluse, i gol effettivamente segnati vengono inseriti nello storico di ciascuna squadra. Questo storico viene gestito come una finestra temporale mobile (*rolling window*) a dimensione fissa: all'inserimento di un nuovo esito, l'osservazione più datata viene scartata, preservando in memoria esclusivamente le ultime tre partite. Questa sequenza viene infine sintetizzata tramite media aritmetica, fornendo lo **stato di forma offensivo** attuale della squadra.

Parallelamente, il sistema estrae i punteggi Elo correnti e crea una **copia operativa isolata**. Questa fotografia della classifica funge da base per i calcoli in tempo reale. Tale isolamento in memoria assicura che qualsiasi esplorazione probabilistica richiesta dall'utente non vada mai a sovrascrivere o inquinare la reale situazione di partenza salvata nel sistema.

È doveroso precisare un aspetto critico legato alla manutenzione di tale file di configurazione. Allo stato attuale dello sviluppo, l'aggiornamento dei valori all'interno dei dizionari `latest_elo` e `partite_reali` viene effettuato **manualmente** al termine di ogni partita della competizione (valori presi dal portale <https://eloratings.net/>). Questa procedura di *data entry*, seppur funzionale ai fini del prototipo, risulta scomoda, dispendiosa in termini di tempo e vulnerabile a potenziali errori di digitazione (*human error*). Tale limitazione strutturale evidenzia la necessità, in ottica di sviluppi futuri, di implementare una *pipeline* di ingestione automatizzata interfacciando il sistema con API RESTful sportive ufficiali. Per un approfondimento sull'architettura proposta per l'automazione di tale flusso di dati, si rimanda all'Appendice B.

Durante la fase di inizializzazione dell'applicazione, il codice esegue il *parsing* del JSON all'interno di un costrutto `try-except`. Tale approccio di *error handling* garantisce la **robustezza dell'app**: in caso di assenza o corruzione del file, il sistema intercetta l'eccezione (`FileNotFoundError`) e invoca le istruzioni per bloccare l'esecuzione con un messaggio diagnostico visibile (`st.error` e `st.stop()`), evitando *crash* incontrollati.

L'utilizzo di *Streamlit* presenta una particolarità tecnica importante: a differenza dei normali server web, il *framework* riesegue l'intero codice dall'inizio alla fine a ogni interazione dell'utente (ad esempio, ogni volta che si seleziona una squadra dal menù). Ricaricare i modelli predittivi dal disco (tramite `joblib.load`) e ricalcolare lo storico delle reti a ogni singolo clic renderebbe l'applicazione eccessivamente lenta e pesante.

Per risolvere questo problema, il sistema utilizza la memorizzazione nella *cache*, avvalendosi di due specifici decoratori:

- `@st.cache_resource`: viene applicato alla funzione che carica le risorse. In questo modo, gli oggetti complessi (come i modelli di *Machine Learning*) vengono caricati in memoria una sola volta al primo avvio, rimanendo subito disponibili per le interazioni successive.
- `@st.cache_data`: viene utilizzato per la funzione che elabora i dati attuali. I dizionari generati vengono salvati in *cache* e restituiti all'istante, costringendo il sistema a ricalcolarli solo se i dati di partenza (il file JSON) vengono modificati.

Grazie a questa gestione della *cache*, le operazioni più pesanti vengono eseguite esclusivamente all'apertura dell'applicazione. Ciò azzera i tempi di caricamento, garantendo all'utente un'esperienza fluida e reattiva, nonostante la complessità del motore di simulazione in *background*.

5.2 Sviluppo dell'Interfaccia Streamlit

L'interfaccia utente è stata concepita per garantire una navigazione intuitiva e una separazione delle funzionalità. A livello strutturale, l'app è stata inizializzata con un *layout* a schermo intero (`layout="wide"`).

Per ottimizzare la resa visiva dell'applicazione, è stato modificato lo stile di *default*. Poiché *Streamlit* genera automaticamente dei *link* di ancoraggio accanto alle intestazioni, si è scelto di nasconderli inserendo del codice CSS personalizzato tramite il parametro `unsafe_allow_html=True`. Questa accortezza garantisce un'interfaccia utente (UI) più pulita e in linea con gli standard delle *dashboard* professionali.

L'architettura di navigazione si sviluppa attorno a una *sidebar* laterale che funge da pannello di controllo. Tramite un componente a scelta singola, l'operatore può scorrere tra le due modalità operative: l'esplorazione di una singola partita e l'esecuzione del simulatore stocastico globale. Tale partizionamento isola le due anime computazionali del progetto, prevenendo il sovraccarico visivo e ottimizzando la fase di interazione dell'utente.

Per una guida visiva all'utilizzo dell'applicazione web e per la consultazione degli screenshot operativi dell'interfaccia, si rimanda all'Appendice C del presente elaborato.

5.3 Modulo 1: Previsore Singolo Match

Il primo modulo incapsula la logica predittiva del sistema al netto della stocasticità, offrendo all'utente la possibilità di calcolare le probabilità esatte per una singola partita. L'interfaccia espone due menù a tendina (`selectbox`) popolati dinamicamente con l'elenco alfabetico delle nazionali tradotte in inglese, aggiornato in tempo reale dallo stato *live* del torneo.

Selezionate le due formazioni, l'algoritmo valida l'input (intercettando e bloccando l'eventuale selezione della medesima squadra in entrambi i campi) e procede al recupero dei rispettivi punteggi Elo e delle medie gol recenti. Da questi valori continui vengono calcolate le differenze (`elo_diff` e `goals_diff`). Contestualmente, il sistema valuta la variabile del fattore campo: se una delle due nazionali figura nella lista predefinita delle nazioni ospitanti (`host_nations`), il parametro `neutral` viene matematicamente impostato a 0.

Il vettore delle *feature* risultante alimenta la fase di inferenza. Il sistema interroga simultaneamente i modelli *Random Forest*, *XGBoost* e *LightGBM*, estraendo le tre distinte distribuzioni di probabilità. Queste vengono aggregate tramite media aritmetica, restituendo le quote definitive del *Soft Ensemble*.

L'interfaccia prevede inoltre un interruttore (*toggle*) per specificare se la partita appartiene a un turno a eliminazione diretta (`is_knockout`). Come analizzato teoricamente nella Sezione 4.4, l'attivazione di questo parametro innesca la ri-proporzione matematica delle probabilità di vittoria delle due squadre (P_A e P_B), forzando a zero l'eventualità del pareggio (P_X).

L'*output* viene infine restituito all'utente suddividendo lo spazio visivo in due sezioni:

- Un blocco testuale riepiloga i parametri numerici calcolati in ingresso (differenza Elo, scarto reti atteso e neutralità del campo). Questa scelta è mirata a garantire totale **trasparenza algoritmica**, permettendo all'operatore di validare le assunzioni del modello prima di leggerne il risultato.
- Una sezione grafica traduce il vettore di probabilità in metriche di sintesi (`st.metric`) e in un grafico a barre generato tramite la libreria *Altair*. Questo diagramma mappa le percentuali dei tre esiti canonici (o di due, nel caso di partita a eliminazione diretta), offrendo una lettura visiva immediata, proporzionale e ben calibrata delle forze in campo.

5.4 Modulo 2: L’“Oracolo” (Simulazione Globale)

L’“Oracolo” rappresenta lo strumento più avanzato dell’applicazione, collegando l’interfaccia utente al motore di simulazione Monte Carlo descritto nel Capitolo 4.

Tramite uno *slider* numerico, l’operatore sceglie il parametro N , ossia il numero complessivo di iterazioni del torneo da simulare (in un intervallo variabile da 100 a 5.000). Questo valore permette di bilanciare la precisione statistica desiderata con il tempo di attesa computazionale.

Premendo il pulsante di avvio, il sistema inizia a generare i vari scenari. Trattandosi di un processo elaborativo intenso, l’applicazione fornisce un *feedback* visivo costante: una barra di avanzamento (`st.progress`) e un contatore testuale si aggiornano progressivamente man mano che 10 cicli vengono completati, indicando all’utente lo stato reale dell’esecuzione.

A simulazione conclusa, l’algoritmo raccoglie l’elenco dei vincitori di ogni singolo torneo generato. Sfruttando l’oggetto `Counter` (importato dal modulo nativo `collections`), il sistema aggrega i risultati e calcola la frequenza percentuale di successo di ogni nazionale, approssimando così la sua reale probabilità di vittoria.

Questi dati vengono infine restituiti tramite un grafico a barre orizzontali, realizzato con la libreria *Altair*. Per garantire la massima leggibilità e non disperdere l’attenzione sulle squadre con probabilità trascurabili, il diagramma filtra e mostra esclusivamente le prime quindici classificate (*Top 15*). L’impatto informativo è rafforzato dall’applicazione di una scala cromatica termica (`scheme='viridis'`): l’intensità del colore cresce proporzionalmente alla percentuale di vittoria, evidenziando in modo immediato le formazioni favorite per la conquista della competizione.

6 Risultati e Discussione

Completato lo sviluppo e il test dell’applicazione, questo capitolo si concentra sull’analisi dei risultati prodotti dal simulatore. L’obiettivo della sezione è interpretare i risultati del modello, per capire in che modo le statistiche iniziali e l’imprevedibilità del tabellone definiscano le probabilità di vittoria per la FIFA World Cup 2026.

6.1 Analisi della Simulazione Globale e Convergenza

Un aspetto cruciale nella valutazione di un modello Monte Carlo è la sensibilità dei risultati al numero di iterazioni (N). Eseguendo il motore stocastico con un campione ridotto (ad esempio $N = 100$, come visibile nei grafici di riferimento in Appendice C), le percentuali di vittoria risultano fortemente instabili. A causa dell’alta varianza statistica intrinseca al calcio, singole esecuzioni da 100 cicli possono produrre risultati diversi in vetta alla classifica, alternando formazioni come Francia, Spagna o Argentina al primo posto in base alle fluttuazioni casuali di quello specifico blocco di simulazioni.

Per mitigare questo rumore stocastico e ottenere stime probabilistiche solide, il modulo “Oracolo” finale è stato avviato impostando un volume massiccio di iterazioni ($N = 5000$). Grazie alla Legge dei Grandi Numeri, un volume così elevato garantisce la convergenza delle frequenze relative verso il reale valore atteso.



Figura 6: Esempio di fluttuazione stocastica con $N = 100$.

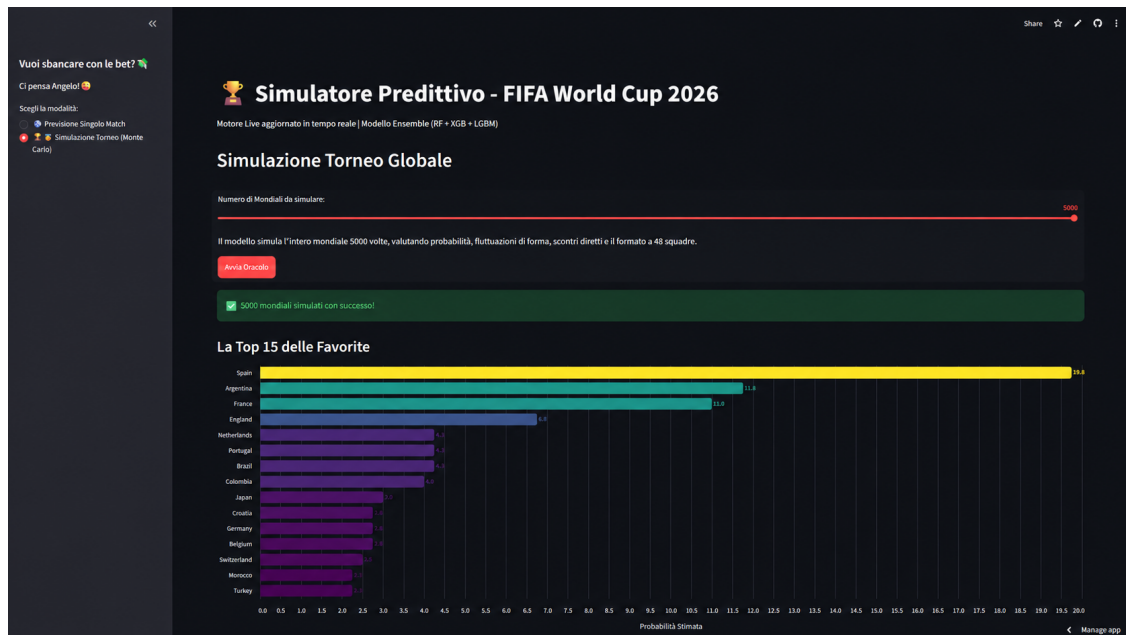


Figura 7: Risultato definitivo della simulazione globale con $N = 5000$ iterazioni.

L'aggregazione dei dati su larga scala mostra un quadro chiaro: la **Spagna** emerge come la nazionale con le maggiori probabilità statistiche di vincere il torneo. Questo primato, già emerso per tutti i numerosi test preliminari più brevi ($N = 1000$), non indica ovviamente che il successo spagnolo sia un esito certo, ma è la conseguenza matematica dei dati di *input* elaborati dal modello *Ensemble*.

Per contestualizzare le metriche di partenza che hanno alimentato il modello, la Tabella 4 riporta i valori estratti dal dataset `elo_ratings_wc2026.csv` (lo stesso impiegato per l'addestramento dei dati e introdotto nella Sezione 2.1) per le prime dieci nazionali al mondo alla vigilia del torneo.

Rank	Squadra	Punteggio Elo
1	Spagna	2165
2	Argentina	2113
3	Francia	2081
4	Inghilterra	2020
5	Brasile	1984
5	Portogallo	1984
7	Colombia	1975
8	Paesi Bassi	1961
9	Ecuador	1933
10	Croazia	1930

Tabella 4: Top 10 nazionali pre-FIFA World Cup 2026, ordinate per punteggio Elo globale.

Il posizionamento della nazionale spagnola al vertice della simulazione è giustificabile analizzando il parametro cardine su cui si fonda l'intera architettura predittiva: il **Ranking Elo di Partenza**. La Spagna inizia infatti il torneo al primo posto assoluto con 2165 punti. Questo valore le garantisce un netto vantaggio statistico sulle dirette inseguitrici, come Argentina (2113) e Francia (2081). Poiché la differenza Elo rappresenta la *feature* più influente per gli algoritmi di *Machine Learning*, questo forte divario iniziale rende la formazione spagnola favorita in quasi tutti i possibili accoppiamenti, massimizzando le sue probabilità di successo all'interno delle migliaia di scenari generati dal motore Monte Carlo.

Alle spalle della favorita, si trovano le formazioni storiche di primissima fascia come **Francia**, **Argentina** e **Brasile** che assorbono la quasi totalità degli esiti residui. Il fatto che le percentuali di vittoria non si concentrino su un'unica squadra dimostra che il programma non si limita a premiare la nazionale con le statistiche migliori, ma tiene conto dell'imprevedibilità di un torneo reale. Un accoppiamento difficile o un risultato a sorpresa, infatti, possono causare l'eliminazione della favorita prima della finale.

6.2 Limiti del Modello

Anche se il modello produce previsioni solide e basate sui dati storici, simulare un torneo reale comporta necessariamente delle semplificazioni. I principali limiti del sistema, che rappresentano anche ottimi spunti per miglioramenti futuri, sono:

- **Differenza Reti nei Gironi:** L'algoritmo calcola le probabilità di vittoria, pareggio o sconfitta (1, X, 2), ma non prevede il risultato esatto (i gol segnati e subiti). Poiché la FIFA usa la differenza reti per decidere chi passa il turno in caso di parità nei gironi, il programma deve usare regole di spareggio semplificate, perdendo un po' di precisione in questa fase.
- **Assenza di Dati sui Singoli Giocatori:** Il sistema valuta le nazionali nel loro complesso, basandosi sulle statistiche di squadra. Non tiene conto, quindi, di eventi fondamentali come l'infortunio improvviso di un campione, le squalifiche, i cambi di allenatore o l'umore dello spogliatoio.
- **Lentezza del Punteggio Elo:** Il *ranking* Elo è molto affidabile, ma ha bisogno di tempo per aggiornarsi. Per questo motivo, potrebbe sottovalutare una squadra emergente spinta da una nuova generazione di talenti o, al contrario, sopravvalutare una nazionale storica che sta attraversando un momento di declino.
- **Fattori Logistici e Ambientali:** I Mondiali del 2026 si giocheranno in tre nazioni enormi (Stati Uniti, Canada e Messico). Il modello sa chi gioca in casa, ma non calcola la stanchezza per i lunghi viaggi o il clima, fattori che influenzeranno molto le prestazioni dei calciatori.

Questi limiti dimostrano la complessità del calcio reale e non compromettono la validità del lavoro svolto. Il simulatore offre infatti una struttura solida, che traccia la strada per i futuri potenziamenti dell'architettura.

7 Conclusioni

Questo progetto ha avuto l'obiettivo di creare un sistema di previsione per la FIFA World Cup 2026, unendo l'intelligenza artificiale (*Machine Learning*) con la statistica sportiva. Il risultato è un simulatore capace di calcolare le probabilità di una singola partita e di prevedere l'intero andamento del torneo, rispettando il nuovo formato a 48 squadre della FIFA.

7.1 Sintesi del Lavoro Svolto

La realizzazione del progetto si è articolata in diverse fasi, coprendo tutti i passaggi chiave dell'analisi dei dati:

- **Preparazione dei Dati e Modello Predittivo:** Il punto di partenza è stato lo storico delle partite internazionali. Sono stati calcolati il punteggio Elo e lo stato di forma recente per misurare matematicamente la forza di ogni squadra. Questi numeri sono serviti per istruire tre algoritmi avanzati (*Random Forest*, *XGBoost* e *LightGBM*), uniti infine in un unico modello (*Ensemble*) per ottenere previsioni più stabili e precise.
- **Simulazione del Torneo:** Per passare dalla singola partita all'intero Mondiale, è stato costruito un simulatore basato sul metodo Monte Carlo. Il programma genera migliaia di scenari possibili, replicando la struttura dei gironi, i criteri di qualificazione e il tabellone

a eliminazione diretta. Questo approccio ha permesso di gestire l'imprevedibilità del calcio, trasformandola in percentuali di vittoria.

- **Creazione dell'Applicazione Web:** Per rendere il modello facile da usare, tutta la logica matematica è stata inserita in un'interfaccia interattiva creata con la libreria *Streamlit*. L'applicazione permette a chiunque di calcolare in tempo reale le probabilità di una partita o di avviare l'"Oracolo" per simulare migliaia di tornei, mostrando i risultati con grafici chiari e intuitivi.

Nel suo complesso, il progetto dimostra come l'intelligenza artificiale possa essere applicata con successo a contesti imprevedibili come le competizioni sportive. Pur senza voler eliminare la naturale incertezza del gioco del calcio, il sistema è riuscito a individuare i fattori che contano di più, trasformandoli in previsioni affidabili e misurabili.

7.2 Sviluppi Futuri e Ottimizzazioni

Il progetto realizzato costituisce una base funzionante, ma lascia spazio a diverse evoluzioni mirate a risolverne le attuali limitazioni. Le direzioni principali per i futuri aggiornamenti riguardano sia il miglioramento della logica algoritmica, sia l'ottimizzazione delle prestazioni informatiche.

Dal punto di vista matematico, per superare le approssimazioni descritte nella Sezione 6.2, l'integrazione di un modello statistico basato sulla distribuzione di Poisson permetterebbe di calcolare la probabilità del risultato esatto, stimando in modo preciso i gol segnati e subiti. Questa implementazione risolverebbe definitivamente il problema dei criteri di spareggio nella fase a gironi. Parallelamente, si prevede l'utilizzo di dati individuali: inserendo statistiche avanzate sui singoli giocatori (come gli *Expected Goals* o lo stato di forma), il sistema affinerrebbe le valutazioni rispetto all'attuale calcolo basato unicamente sulla forza complessiva della nazionale.

Sotto il profilo tecnico, l'attenzione principale è rivolta alle prestazioni del motore Monte Carlo. Attualmente, quando si imposta un numero elevato di simulazioni (ad esempio $N = 5000$), i tempi di attesa diventano inevitabilmente lunghi a causa dell'elaborazione sequenziale (qualche ora). Poiché ogni iterazione del torneo è un processo indipendente dagli altri, la simulazione rientra nella categoria dei problemi *embarrassingly parallel*. Di conseguenza, un'ottimizzazione architetturale passerebbe per lo sfruttamento del calcolo parallelo su CPU multi-core. Implementando librerie di multiprocessing (come *joblib*), sarebbe possibile distribuire i carichi di lavoro sui diversi thread logici del processore, riducendo drasticamente i tempi di esecuzione e rendendo il sistema reattivo per un utilizzo su vasta scala.

Infine, come già introdotto nella Sezione 5.1, l'applicazione potrebbe essere collegata a servizi web esterni (API) per scaricare in automatico i risultati ufficiali delle partite appena concluse. In questo modo, il simulatore ricalcolerebbe le probabilità di vittoria dell'intero torneo in tempo reale, aggiornando le stime giorno dopo giorno durante lo svolgimento del Mondiale.

A Il Sistema di Rating Elo nel Calcio

Il punteggio Elo, originariamente sviluppato dal fisico ungherese Arpad Elo negli anni '60 per valutare la forza relativa dei giocatori di scacchi, è stato successivamente adattato al calcio internazionale [2]. A differenza dei classici ranking basati su accumulo a finestre temporali fisse, il sistema Elo si fonda su un modello statistico *zero-sum*, capace di tracciare dinamicamente la forza intrinseca di un'entità competitiva.

A.1 La Formulazione Matematica

Il principio fondante del sistema afferma che la differenza di punteggio tra due sfidanti determini la probabilità di vittoria attesa. Se la squadra A possiede un rating R_A e la squadra B un rating R_B , il punteggio atteso (o probabilità di vittoria) per la squadra A viene calcolato tramite la seguente funzione logistica:

$$E_A = \frac{1}{1 + 10^{\frac{R_B - R_A}{400}}}$$

Di conseguenza, il punteggio atteso per la squadra B sarà il complemento $E_B = 1 - E_A$. Il denominatore 400 rappresenta una costante di scala che tara la “sensibilità” del modello: una differenza esatta di 400 punti implica che la squadra favorita goda di circa il 90% di probabilità di vittoria teorica.

Al termine dell'incontro, il rating di ciascuna squadra viene aggiornato calcolando la discrepanza tra l'aspettativa matematica e la realtà, secondo la regola:

$$R'_A = R_A + K \cdot (S_A - E_A) \quad (2)$$

Dove le componenti indicano:

- R'_A : il nuovo rating aggiornato della squadra A .
- K : il K -factor, una costante moltiplicativa che funge da *learning rate*, determinando la volatilità del ranking e la velocità di reazione del sistema ai nuovi risultati.
- S_A : il risultato effettivo (*Score*) della partita (1 per la vittoria, 0 per la sconfitta, 0.5 per il pareggio).
- E_A : il risultato teorico atteso prima del fischio d'inizio.

Se una squadra ottiene un risultato migliore del previsto ($S_A > E_A$), il suo rating aumenta; se ottiene un risultato peggiore, diminuisce. Essendo un sistema a somma zero, i punti guadagnati dalla formazione che over-performa vengono matematicamente sottratti al rating dell'avversaria.

A.2 Adattamenti Teorici per il Calcio

L'applicazione di una formula scacchistica a uno sport di squadra stocastico richiede specifici aggiustamenti strutturali della teoria di base. Le principali differenze nell'impiego calcistico riguardano:

1. **Mappatura del Pareggio:** Mentre negli scacchi la patta è un esito formale, nel calcio il pareggio possiede una precisa incidenza statistica. Il sistema modella questo evento assegnando $S = 0.5$ a entrambe le squadre, un'operazione che penalizza lievemente la squadra favorita (che si aspettava un E_A alto) e premia l'*underdog* in caso di pareggio inatteso.
2. **Dinamicità del K-Factor:** Per riflettere la disparità di importanza tra gli incontri, nel calcio il fattore K non è quasi mai statico. Viene tipicamente scalato in base al prestigio del torneo (es. amichevole vs Mondiale) e, in alcune implementazioni avanzate, viene ulteriormente moltiplicato per l'indice di *Goal Difference* (lo scarto reti), al fine di premiare le vittorie nette rispetto a quelle di misura.
3. **Vantaggio Casalingo (*Home Advantage*):** A livello teorico, per assorbire il *bias* statistico del fattore campo, i sistemi Elo calcistici prevedono spesso l'addizione di una costante fissa (solitamente stimata tra gli 80 e i 100 punti) al rating di partenza della squadra ospitante esclusivamente durante il calcolo dell'aspettativa pre-match (E_A).

B Automazione dell'Aggiornamento Dati tramite API

Come discusso nella Sezione 5.1, l'attuale sistema richiede di aggiornare a mano il file `config_mondiale.json` per inserire i risultati delle partite e aggiornare i punteggi Elo. Anche se questa soluzione è accettabile per un prototipo, alla lunga risulta poco pratica e aumenta il rischio di commettere errori di digitazione.

Per un'applicazione definitiva, la soluzione ideale è sostituire l'inserimento manuale con un processo completamente automatico (una *pipeline* ETL: *Extract, Transform, Load*), collegando il sistema alle API fornite dai servizi ufficiali di statistiche sportive.

L'infrastruttura software per questa integrazione si dividerebbe in tre passaggi sequenziali:

1. **Estrazione dei Dati (*Extract*):** Un programma schedato per eseguirsi a intervalli regolari avvia uno *script* Python. Questo *script* contatta i server esterni per scaricare in automatico i risultati dei match appena conclusi.
2. **Elaborazione dei Dati (*Transform*):** I dati grezzi appena scaricati vengono letti e filtrati (*parsing*). Il sistema estrae le informazioni utili (nomi delle squadre e gol segnati) per aggiornare lo storico recente delle reti. Subito dopo, il risultato esatto viene passato alla formula matematica dell'Elo per calcolare la nuova classifica.
3. **Salvataggio (*Load*):** I nuovi dizionari aggiornati (`partite_reali` e `latest_elo`) vengono salvati nuovamente in formato JSON. Il file di configurazione viene così sovrascritto in modo sicuro, rendendo le nuove informazioni istantaneamente disponibili all'interfaccia Streamlit per le simulazioni successive.

Sviluppare questo modulo renderebbe il simulatore completamente autonomo. Le statistiche live delle partite sarebbero sempre allineate alla realtà in tempi rapidissimi, eliminando del tutto la necessità di interventi umani e rendendo il prodotto software molto più solido e professionale.

C Guida all'utilizzo dell'applicazione web

Questa appendice fornisce una guida visiva e operativa all'interfaccia dell'applicazione sviluppata con *Streamlit*, illustrando i passaggi necessari per interrogare i due moduli principali del sistema predittivo.

Struttura Generale e Navigazione

L'applicazione si presenta con un *layout* a schermo intero. Sul lato sinistro della finestra è posizionata una *sidebar* di navigazione, che funge da pannello di controllo principale. Tramite un selettore a spunta, l'utente può decidere quale strumento avviare: la previsione del singolo match o la simulazione globale Monte Carlo.

Utilizzo del Modulo 1: Previsore Singolo Match

Selezionando la prima modalità, l'interfaccia carica gli strumenti di inferenza deterministica. Come illustrato nella Figura 8, l'utente viene accolto da una schermata di inserimento dati.

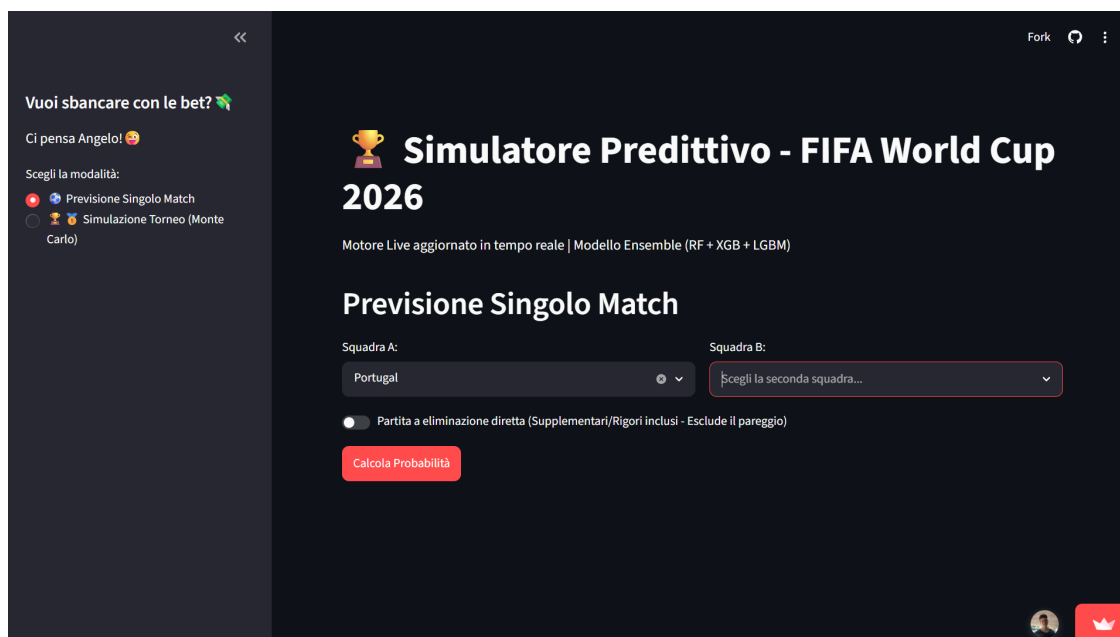


Figura 8: Schermata iniziale del modulo di previsione.

Per ottenere una previsione, è necessario seguire questi passaggi:

1. **Selezione delle squadre:** Utilizzare i due menù a tendina centrali per scegliere la formazione di casa e quella in trasferta.
2. **Fase del torneo:** Attivare o disattivare l'interruttore per le partite a eliminazione diretta, a seconda della fase della competizione che si desidera simulare.
3. **Calcolo:** Premere il pulsante rosso di conferma per innescare l'algoritmo.

Una volta elaborati i dati, il sistema espone i risultati operando in totale trasparenza. La Figura 9 mostra l’output per un ipotetico scontro di fase a gironi tra Portogallo e Spagna. Il riquadro verde in alto riepiloga i parametri matematici calcolati in tempo reale (divario Elo, stato di forma recente e vantaggio casalingo) che sono stati passati in *input* all’Ensemble. Subito sotto, il grafico a barre generato tramite *Altair* distribuisce le probabilità sui tre esiti classici.

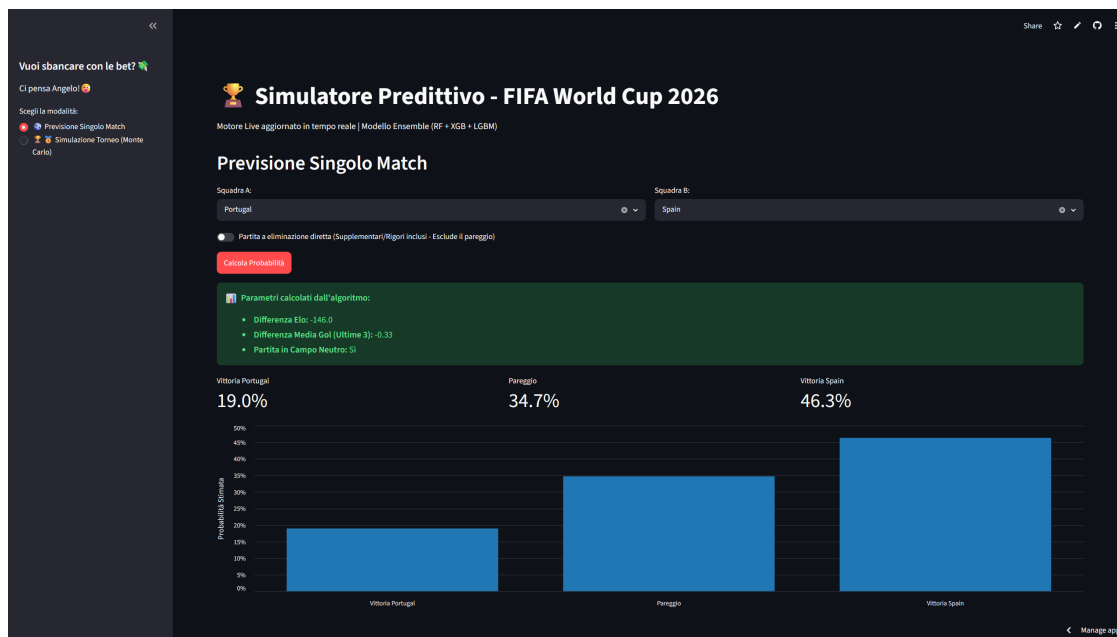


Figura 9: Risultato dell’inferenza base (fase a gironi con possibilità di pareggio) per il match Portogallo-Spagna.

Nella Figura 10, a parità di parametri di partenza, l’algoritmo inibisce la possibilità del pareggio e ri-proporziona matematicamente le probabilità di successo tra le due formazioni.

Utilizzo del Modulo 2: L’“Oracolo”

Selezionando la seconda modalità dalla *sidebar*, l’applicazione carica la simulazione globale. Il processo operativo è diviso in tre fasi distinte.

Nella fase di **impostazione** (Figura 11), l’utente regola lo *slider* numerico per definire il numero totale di iterazioni Monte Carlo da eseguire (N). Questo parametro permette di bilanciare la precisione statistica con i tempi di elaborazione della macchina. Una volta impostato il valore (es. 1800 simulazioni), la pressione del pulsante “Avvia Oracolo” innesca il motore stocastico.

Trattandosi di un’operazione computazionalmente densa, il sistema è stato dotato di un **sistema di monitoraggio** per migliorare l’esperienza utente. Come mostrato nella Figura 12, durante l’esecuzione appare una *progress bar* accompagnata da un contatore dinamico che aggiorna l’operatore sullo stato di avanzamento in tempo reale ogni 10 simulazioni.

Al termine delle iterazioni, l’interfaccia espone i **risultati** (Figura 13). Il successo dell’operazione viene notificato da un banner testuale, seguito dal grafico a barre generato

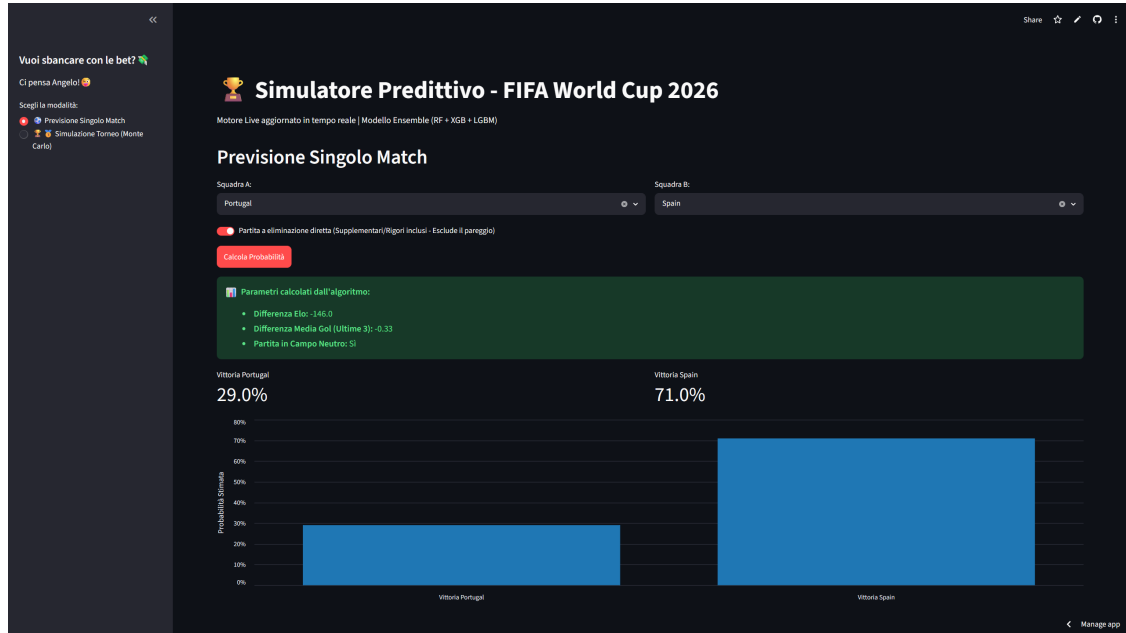


Figura 10: Ricalcolo delle probabilità per il medesimo match attivando il vincolo dell'eliminazione diretta.

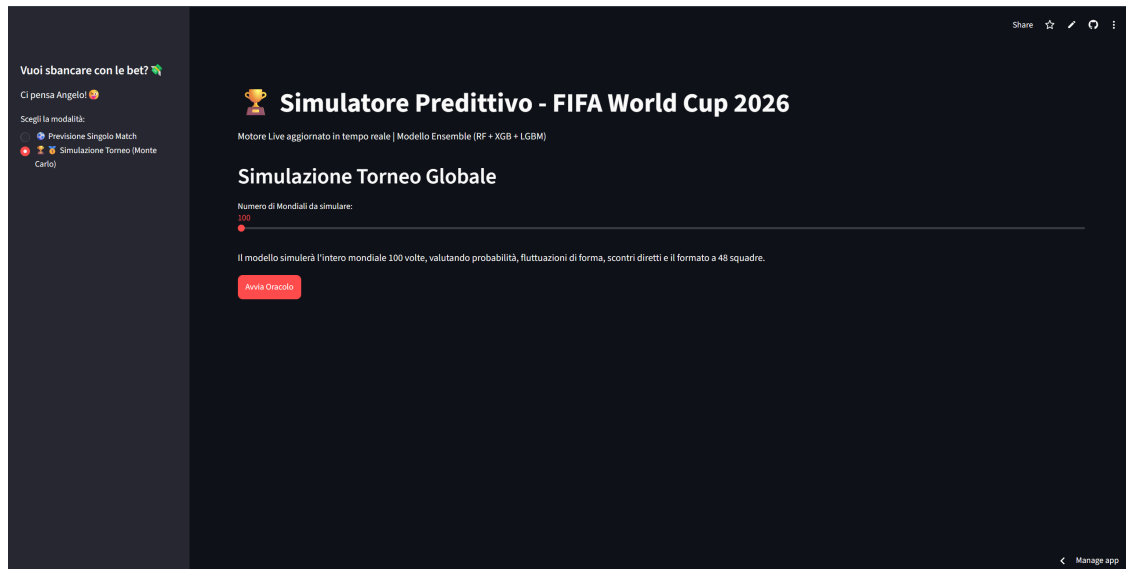


Figura 11: Schermata di impostazione dell'“Oracolo”, con lo slider per la selezione del numero di iterazioni desiderate.

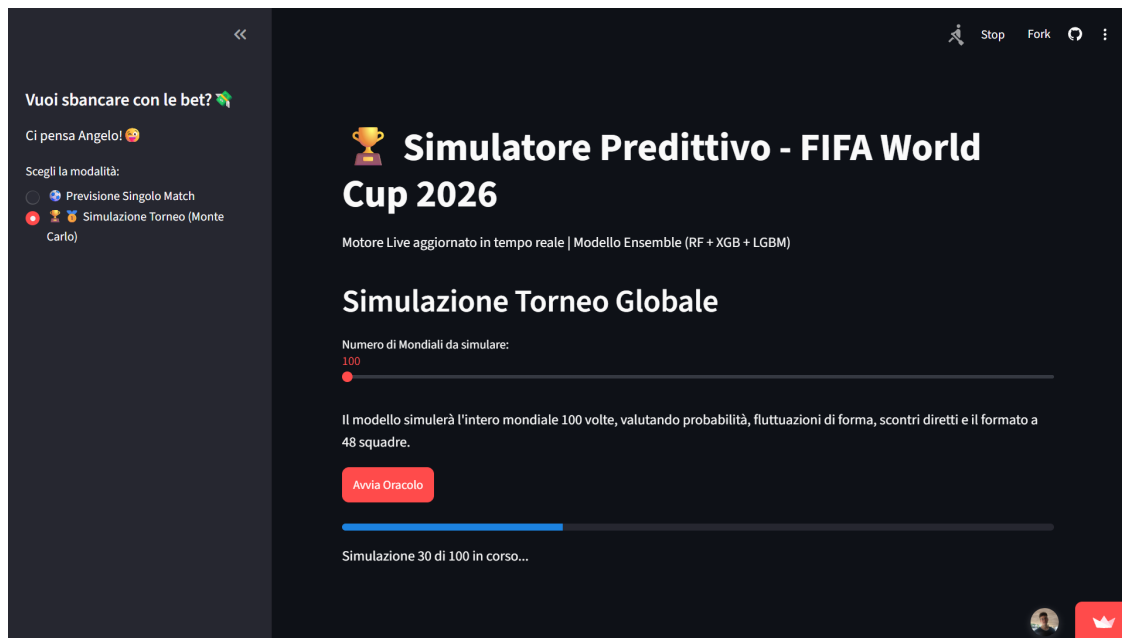


Figura 12: Feedback visivo durante l'esecuzione del ciclo Monte Carlo, con barra di progresso e stato di avanzamento dinamico.

dalla libreria *Altair*. Per garantire una maggiore chiarezza informativa, il diagramma filtra e mostra esclusivamente la *Top 15* delle nazionali favorite. L'utilizzo della scala cromatica *viridis* (che sfuma dal giallo per i valori più alti al viola scuro per quelli inferiori) permette una mappatura visiva immediata delle gerarchie di forza calcolate dal modello.



Figura 13: L'output finale della simulazione globale. Il grafico a barre evidenzia la probabilità percentuale di vittoria per la Top 15 delle nazionali.

Riferimenti bibliografici

- [1] Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, and Jonathan Taylor. *An Introduction to Statistical Learning: with Applications in Python*. Springer Nature, 2023.
- [2] Jan Lasek, Zoltán Szilávik, and Sandjai Bhulai. The predictive power of ranking systems in association football. *International Journal of Applied Pattern Recognition*, 1(1):27–46, 2013.